

```
0001 # Source file: backend/src/backend/__init__.py
0002 def main() -> None:
0003     print("Hello from backend!")
0004 # Source file: backend/src/backend/resources.py
0005 import sys
0006 from pathlib import Path
0007 def application_root() -> Path:
0008     bundled_root = getattr(sys, "_MEIPASS", None)
0009     if bundled_root:
0010         return Path(bundled_root)
0011     return Path(__file__).resolve().parents[3]
0012 def frontend_directory() -> Path:
0013     root = application_root()
0014     if getattr(sys, "frozen", False):
0015         return root / "frontend"
0016     return root / "frontend" / "dist"
0017 # Source file: backend/src/backend/desktop.py
0018 import ctypes
0019 import logging
0020 import os
0021 import socket
0022 import sys
0023 import threading
0024 import time
0025 from pathlib import Path
0026 from urllib.error import URLError
0027 from urllib.request import urlopen
0028 import uvicorn
0029 import webview
0030 from backend.main import app
0031 from backend.resources import frontend_directory
0032 APP_NAME = "DataEX-G"
0033 APP_TITLE = "DataEX-G"
0034 MUTEX_NAME = "Local\\GGzzzW.DataEX-G"
0035 LOGGER = logging.getLogger(__name__)
0036 def local_data_directory() -> Path:
0037     base = Path(os.environ.get("LOCALAPPDATA", Path.home() / "AppData" / "Local"))
0038     directory = base / APP_NAME
0039     directory.mkdir(parents=True, exist_ok=True)
0040     return directory
0041 def configure_logging() -> None:
0042     log_directory = local_data_directory() / "logs"
0043     log_directory.mkdir(parents=True, exist_ok=True)
0044     logging.basicConfig(
0045         filename=log_directory / "desktop.log",
0046         level=logging.INFO,
0047         format="%(asctime)s %(levelname)s %(name)s %(message)s",
0048         encoding="utf-8",
0049     )
0050 def show_error(message: str) -> None:
```

```

0051     LOGGER.error(message)
0052     if sys.platform == "win32":
0053         ctypes.windll.user32.MessageBoxW(0, message, APP_TITLE, 0x10)
0054     else:
0055         print(message, file=sys.stderr)
0056 def acquire_single_instance() -> int | None:
0057     if sys.platform != "win32":
0058         return 1
0059     handle = ctypes.windll.kernel32.CreateMutexW(None, False, MUTEX_NAME)
0060     if not handle:
0061         return None
0062     if ctypes.windll.kernel32.GetLastError() == 183:
0063         ctypes.windll.kernel32.CloseHandle(handle)
0064         return None
0065     return int(handle)
0066 def release_single_instance(handle: int | None) -> None:
0067     if sys.platform == "win32" and handle:
0068         ctypes.windll.kernel32.CloseHandle(handle)
0069 def create_server() -> tuple[uvicorn.Server, socket.socket, int]:
0070     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
0071     server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
0072     server_socket.bind(("127.0.0.1", 0))
0073     server_socket.listen(2048)
0074     port = int(server_socket.getsockname()[1])
0075     config = uvicorn.Config(
0076         app,
0077         host="127.0.0.1",
0078         port=port,
0079         log_level="warning",
0080         access_log=False,
0081     )
0082     return uvicorn.Server(config), server_socket, port
0083 def wait_for_server(url: str, timeout_seconds: float = 15) -> None:
0084     deadline = time.monotonic() + timeout_seconds
0085     while time.monotonic() < deadline:
0086         try:
0087             with urlopen(f"{url}/health", timeout=1) as response:
0088                 if response.status == 200:
0089                     return
0090         except (OSError, URLError):
0091             time.sleep(0.1)
0092     raise RuntimeError("本地分析服务启动超时。")
0093 def run_smoke_test(url: str) -> None:
0094     with urlopen(f"{url}/health", timeout=5) as response:
0095         if response.status != 200:
0096             raise RuntimeError("健康检查失败。")
0097     with urlopen(url, timeout=5) as response:
0098         page = response.read(4096)
0099         if response.status != 200 or b"<html" not in page.lower():
0100             raise RuntimeError("Vue 首页没有正确打包。")

```

```

0101 def main() -> int:
0102     configure_logging()
0103     instance_handle = acquire_single_instance()
0104     if instance_handle is None:
0105         show_error("DataEX-G 已经在运行。")
0106         return 1
0107     if not (frontend_directory() / "index.html").is_file():
0108         show_error("没有找到前端页面，请先运行前端生产构建。")
0109         release_single_instance(instance_handle)
0110         return 1
0111     server, server_socket, port = create_server()
0112     server_thread = threading.Thread(
0113         target=server.run,
0114         kwargs={"sockets": [server_socket]},
0115         name="local-api",
0116         daemon=True,
0117     )
0118     server_thread.start()
0119     url = f"http://127.0.0.1:{port}"
0120     try:
0121         wait_for_server(url)
0122         if os.environ.get("DATA_ANALYSIS_DESKTOP_SMOKE_TEST") == "1":
0123             run_smoke_test(url)
0124             return 0
0125         webview.create_window(
0126             APP_TITLE,
0127             url,
0128             width=1280,
0129             height=820,
0130             min_size=(960, 640),
0131             background_color="#edf2ed",
0132         )
0133         webview.start(
0134             debug=False,
0135             private_mode=False,
0136             storage_path=str(local_data_directory() / "webview"),
0137         )
0138         return 0
0139     except (OSError, RuntimeError) as exc:
0140         show_error(f"程序启动失败: {exc}")
0141         return 1
0142     finally:
0143         server.should_exit = True
0144         server_thread.join(timeout=5)
0145         server_socket.close()
0146         release_single_instance(instance_handle)
0147 if __name__ == "__main__":
0148     raise SystemExit(main())
0149 # Source file: backend/src/backend/quality.py
0150 import re

```

```

0151 from collections import Counter, defaultdict
0152 from datetime import date, datetime
0153 from numbers import Number
0154 import pandas as pd
0155 NUMBER_PATTERN = re.compile(r"^[+-]?(\d+(\.\d*)?|\.\d+)(?[eE][+-]?\d+)?$")
0156 DATE_PATTERNS = (
0157     re.compile(r"^\d{4}[-/]\d{1,2}[-/]\d{1,2}([T].*)?$"),
0158     re.compile(r"^\d{1,2}[-/]\d{1,2}[-/]\d{4}([T].*)?$"),
0159     re.compile(r"^\d{4}年\d{1,2}月\d{1,2}日([T].*)?$"),
0160 )
0161 def is_missing(value: object) -> bool:
0162     if isinstance(value, str):
0163         return not value.strip()
0164     return bool(pd.isna(value))
0165 def detect_value_type(value: object) -> str:
0166     if isinstance(value, bool):
0167         return "boolean"
0168     if isinstance(value, Number):
0169         return "number"
0170     if isinstance(value, (datetime, date, pd.Timestamp)):
0171         return "datetime"
0172     if not isinstance(value, str):
0173         return "text"
0174     normalized = value.strip()
0175     if normalized.lower() in ["true", "false"]:
0176         return "boolean"
0177     if NUMBER_PATTERN.fullmatch(normalized):
0178         return "number"
0179     if any(pattern.fullmatch(normalized) for pattern in DATE_PATTERNS):
0180         return "datetime"
0181     return "text"
0182 def analyze_column(name: object, series: pd.Series) -> dict[str, object]:
0183     type_counts: Counter[str] = Counter()
0184     examples: defaultdict[str, list[str]] = defaultdict(list)
0185     missing_count = 0
0186     whitespace_row_numbers: list[int] = []
0187     line_break_row_numbers: list[int] = []
0188     for row_number, value in enumerate(series.tolist(), start=1):
0189         if is_missing(value):
0190             missing_count += 1
0191             continue
0192         if isinstance(value, str):
0193             if value != value.strip():
0194                 whitespace_row_numbers.append(row_number)
0195             if "\r" in value or "\n" in value:
0196                 line_break_row_numbers.append(row_number)
0197         value_type = detect_value_type(value)
0198         type_counts[value_type] += 1
0199         if len(examples[value_type]) < 3:
0200             examples[value_type].append(str(value))

```

```

0201     detected_types = [
0202         {
0203             "type": value_type,
0204             "count": count,
0205             "examples": examples[value_type],
0206         }
0207         for value_type, count in sorted(
0208             type_counts.items(), key=lambda item: (-item[1], item[0])
0209         )
0210     ]
0211     row_count = len(series)
0212     return {
0213         "name": str(name),
0214         "pandas_dtype": str(series.dtype),
0215         "missing_count": missing_count,
0216         "missing_ratio": round(missing_count / row_count, 4) if row_count else 0.0,
0217         "detected_types": detected_types,
0218         "mixed_types": len(type_counts) > 1,
0219         "whitespace_count": len(whitespace_row_numbers),
0220         "whitespace_row_numbers": whitespace_row_numbers,
0221         "line_break_count": len(line_break_row_numbers),
0222         "line_break_row_numbers": line_break_row_numbers,
0223     }
0224 def build_quality_report(dataframe: pd.DataFrame) -> dict[str, object]:
0225     columns = [
0226         analyze_column(name, dataframe.iloc[:, position])
0227         for position, name in enumerate(dataframe.columns)
0228     ]
0229     duplicate_mask = dataframe.duplicated(keep=False)
0230     return {
0231         "missing_cell_count": sum(int(column["missing_count"]) for column in columns),
0232         "whitespace_cell_count": sum(
0233             int(column["whitespace_count"]) for column in columns
0234         ),
0235         "line_break_cell_count": sum(
0236             int(column["line_break_count"]) for column in columns
0237         ),
0238         "duplicate_row_count": int(dataframe.duplicated().sum()),
0239         "duplicate_row_numbers": [
0240             position + 1
0241             for position, is_duplicate in enumerate(duplicate_mask.tolist())
0242             if is_duplicate
0243         ],
0244         "columns": columns,
0245     }
0246 # Source file: backend/src/backend/cleaning.py
0247 import re
0248 from io import BytesIO
0249 from typing import Literal
0250 import pandas as pd

```

```

0251 from backend.quality import is_missing
0252 MissingAction = Literal["none", "drop_rows", "extract_rows", "fill_zero"]
0253 ExportFormat = Literal["csv", "xlsx"]
0254 ExportTable = Literal["cleaned", "extracted"]
0255 StandardizationMethod = Literal["none", "min_max", "z_score"]
0256 LINE_BREAK_PATTERN = re.compile(r"[\r\n]+")
0257 class StandardizationError(ValueError):
0258     pass
0259 def build_missing_mask(dataframe: pd.DataFrame) -> pd.DataFrame:
0260     return dataframe.map(is_missing)
0261 def clean_text_value(
0262     value: object,
0263     *,
0264     trim_whitespace: bool,
0265     remove_line_breaks: bool,
0266 ) -> object:
0267     if not isinstance(value, str):
0268         return value
0269     cleaned = value.strip() if trim_whitespace else value
0270     if remove_line_breaks:
0271         cleaned = LINE_BREAK_PATTERN.sub(" ", cleaned)
0272     return cleaned
0273 def clean_text_dataframe(
0274     dataframe: pd.DataFrame,
0275     *,
0276     trim_whitespace: bool,
0277     remove_line_breaks: bool,
0278 ) -> tuple[pd.DataFrame, int]:
0279     changed_cell_count = 0
0280     def transform(value: object) -> object:
0281         nonlocal changed_cell_count
0282         cleaned = clean_text_value(
0283             value,
0284             trim_whitespace=trim_whitespace,
0285             remove_line_breaks=remove_line_breaks,
0286         )
0287         if isinstance(value, str) and cleaned != value:
0288             changed_cell_count += 1
0289         return cleaned
0290     return dataframe.map(transform), changed_cell_count
0291 def standardize_dataframe(
0292     dataframe: pd.DataFrame,
0293     *,
0294     method: StandardizationMethod,
0295     columns: list[str],
0296 ) -> tuple[pd.DataFrame, list[dict[str, object]]]:
0297     standardized = dataframe.copy()
0298     statistics: list[dict[str, object]] = []
0299     if method == "none":
0300         return standardized, statistics

```

```

0301     if not columns:
0302         raise StandardizationError("Select at least one numeric column to standardize.")
0303     for column in columns:
0304         if column not in standardized.columns:
0305             raise StandardizationError(f"Column '{column}' does not exist.")
0306         original = standardized[column]
0307         numeric = pd.to_numeric(original, errors="coerce")
0308         invalid_mask = ~original.map(is_missing) & numeric.isna()
0309         if invalid_mask.any():
0310             raise StandardizationError(
0311                 f"Column '{column}' contains non-numeric values and cannot be standardized."
0312             )
0313         valid = numeric.dropna()
0314         if valid.empty:
0315             raise StandardizationError(
0316                 f"Column '{column}' has no numeric values to standardize."
0317             )
0318         minimum = float(valid.min())
0319         maximum = float(valid.max())
0320         mean = float(valid.mean())
0321         standard_deviation = float(valid.std(ddof=0))
0322         if method == "min_max":
0323             denominator = maximum - minimum
0324             transformed = (
0325                 numeric * 0 if denominator == 0 else (numeric - minimum) / denominator
0326             )
0327         else:
0328             transformed = (
0329                 numeric * 0
0330                 if standard_deviation == 0
0331                 else (numeric - mean) / standard_deviation
0332             )
0333         standardized[column] = transformed.round(6)
0334         statistics.append(
0335             {
0336                 "column": column,
0337                 "minimum": minimum,
0338                 "maximum": maximum,
0339                 "mean": round(mean, 6),
0340                 "standard_deviation": round(standard_deviation, 6),
0341             }
0342         )
0343     return standardized, statistics
0344 def clean_dataframe(
0345     dataframe: pd.DataFrame,
0346     *,
0347     missing_action: MissingAction,
0348     trim_whitespace: bool,
0349     remove_line_breaks: bool,
0350     standardization_method: StandardizationMethod = "none",

```

```

0351     standardization_columns: list[str] | None = None,
0352 ) -> tuple[pd.DataFrame, pd.DataFrame, dict[str, object]]:
0353     missing_mask = build_missing_mask(dataframe)
0354     affected_rows = missing_mask.any(axis=1)
0355     extracted_row_numbers: list[int] = []
0356     extracted = dataframe.iloc[0:0].copy()
0357     cleaned = dataframe.copy()
0358     if missing_action == "drop_rows":
0359         cleaned = dataframe.loc[~affected_rows].copy()
0360     elif missing_action == "extract_rows":
0361         extracted = dataframe.loc[affected_rows].copy()
0362         extracted_row_numbers = [
0363             position + 1
0364             for position, has_missing in enumerate(affected_rows.tolist())
0365             if has_missing
0366         ]
0367         cleaned = dataframe.loc[~affected_rows].copy()
0368     elif missing_action == "fill_zero":
0369         cleaned = dataframe.mask(missing_mask, 0)
0370     text_changed_cell_count = 0
0371     if trim_whitespace or remove_line_breaks:
0372         cleaned, cleaned_change_count = clean_text_dataframe(
0373             cleaned,
0374             trim_whitespace=trim_whitespace,
0375             remove_line_breaks=remove_line_breaks,
0376         )
0377         extracted, extracted_change_count = clean_text_dataframe(
0378             extracted,
0379             trim_whitespace=trim_whitespace,
0380             remove_line_breaks=remove_line_breaks,
0381         )
0382         text_changed_cell_count = cleaned_change_count + extracted_change_count
0383     cleaned = cleaned.reset_index(drop=True)
0384     extracted = extracted.reset_index(drop=True)
0385     cleaned, standardization_statistics = standardize_dataframe(
0386         cleaned,
0387         method=standardization_method,
0388         columns=standardization_columns or [],
0389     )
0390     return (
0391         cleaned,
0392         extracted,
0393         {
0394             "missing_action": missing_action,
0395             "missing_affected_row_count": int(affected_rows.sum()),
0396             "text_changed_cell_count": text_changed_cell_count,
0397             "extracted_row_numbers": extracted_row_numbers,
0398             "standardization_method": standardization_method,
0399             "standardized_columns": standardization_columns or [],
0400             "standardization_statistics": standardization_statistics,

```

```

0401         },
0402     )
0403 def export_csv(dataframe: pd.DataFrame) -> bytes:
0404     return dataframe.to_csv(index=False, lineterminator="\n").encode("utf-8-sig")
0405 def export_xlsx(cleaned: pd.DataFrame, extracted: pd.DataFrame) -> bytes:
0406     output = BytesIO()
0407     with pd.ExcelWriter(output, engine="openpyxl") as writer:
0408         cleaned.to_excel(writer, sheet_name="cleaned_data", index=False)
0409         if not extracted.empty:
0410             extracted.to_excel(writer, sheet_name="missing_data", index=False)
0411     return output.getvalue()
0412 # Source file: backend/src/backend/diagnostics.py
0413 import math
0414 import warnings
0415 from collections.abc import Iterable
0416 import numpy as np
0417 import pandas as pd
0418 from statsmodels.stats.outliers_influence import variance_inflation_factor
0419 def _finite_float(value: object) -> float | None:
0420     converted = float(value)
0421     return converted if math.isfinite(converted) else None
0422 def _append_unique(messages: list[str], message: str) -> None:
0423     if message and message not in messages:
0424         messages.append(message)
0425 def build_design_diagnostics(
0426     dataframe: pd.DataFrame, columns: list[str]
0427 ) -> dict[str, object]:
0428     values = dataframe[columns].to_numpy(dtype=float)
0429     parameter_count = len(columns) + 1
0430     diagnostic_warnings: list[str] = []
0431     standard_deviations = np.std(values, axis=0, ddof=0)
0432     nonzero_scales = standard_deviations[standard_deviations > 0]
0433     scale_ratio = (
0434         float(np.max(nonzero_scales) / np.min(nonzero_scales))
0435         if len(nonzero_scales) > 1
0436         else 1.0
0437     )
0438     if scale_ratio >= 1000:
0439         _append_unique(
0440             diagnostic_warnings,
0441             "自变量的数值尺度相差超过 1000 倍，建议仅对自变量进行 Z-score 标准化。",
0442         )
0443     standardized = np.divide(
0444         values - np.mean(values, axis=0),
0445         standard_deviations,
0446         out=np.zeros_like(values, dtype=float),
0447         where=standard_deviations != 0,
0448     )
0449     standardized_design = np.column_stack((np.ones(len(values)), standardized))
0450     raw_design = np.column_stack((np.ones(len(values)), values))

```

```

0451 rank = int(np.linalg.matrix_rank(standardized_design))
0452 condition_number = float(np.linalg.cond(standardized_design))
0453 raw_condition_number = float(np.linalg.cond(raw_design))
0454 if rank < parameter_count:
0455     _append_unique(
0456         diagnostic_warnings,
0457         "设计矩阵不满秩，部分自变量可能完全重复或可由其他变量线性组合得到。",
0458     )
0459 elif condition_number >= 30:
0460     _append_unique(
0461         diagnostic_warnings,
0462         "标准化后的设计矩阵条件数较高，结果可能受到多重共线性影响。",
0463     )
0464 if raw_condition_number >= 1_000_000:
0465     _append_unique(
0466         diagnostic_warnings,
0467         "原始尺度条件数超过 100 万，模型可能发生数值不稳定；建议仅标准化自变量。",
0468     )
0469 vif_rows = []
0470 design_with_constant = np.column_stack((np.ones(len(values)), values))
0471 for index, column in enumerate(columns, start=1):
0472     try:
0473         with warnings.catch_warnings():
0474             warnings.simplefilter("ignore", category=RuntimeWarning)
0475             vif_value = float(
0476                 variance_inflation_factor(design_with_constant, index)
0477             )
0478         except (ValueError, np.linalg.LinAlgError, ZeroDivisionError):
0479             vif_value = math.inf
0480         vif_rows.append({"column": column, "vif": _finite_float(vif_value)})
0481 finite_vifs = [row["vif"] for row in vif_rows if row["vif"] is not None]
0482 max_vif = max(finite_vifs, default=None)
0483 infinite_vif = any(row["vif"] is None for row in vif_rows)
0484 if infinite_vif or (max_vif is not None and max_vif >= 10):
0485     _append_unique(
0486         diagnostic_warnings,
0487         "检测到严重多重共线性 (VIF ≥ 10 或无穷大)，系数和显著性可能不稳定。",
0488     )
0489 elif max_vif is not None and max_vif >= 5:
0490     _append_unique(
0491         diagnostic_warnings,
0492         "检测到较强多重共线性 (VIF ≥ 5)，建议检查变量选择。",
0493     )
0494 return {
0495     "converged": True,
0496     "valid_inference": rank == parameter_count,
0497     "rank": rank,
0498     "parameter_count": parameter_count,
0499     "condition_number": _finite_float(condition_number),
0500     "raw_condition_number": _finite_float(raw_condition_number),

```

```

0501         "scale_ratio": _finite_float(scale_ratio),
0502         "max_vif": max_vif,
0503         "vif": vif_rows,
0504         "warnings": diagnostic_warnings,
0505     }
0506 def finalize_model_diagnostics(
0507     diagnostics: dict[str, object],
0508     *,
0509     converged: bool,
0510     inference_arrays: Iterable[object],
0511     captured_warnings: Iterable[str] = (),
0512     model_warning: str | None = None,
0513 ) -> dict[str, object]:
0514     messages = list(diagnostics["warnings"])
0515     for message in captured_warnings:
0516         _append_unique(messages, f"模型运行警告: {message}")
0517     if model_warning:
0518         _append_unique(messages, f"模型警告: {model_warning}")
0519     if not converged:
0520         _append_unique(
0521             messages,
0522             "模型未收敛; 当前系数、标准误、p 值和置信区间不能作为可靠结论。",
0523         )
0524     finite_inference = True
0525     for values in inference_arrays:
0526         try:
0527             if not np.isfinite(np.asarray(values, dtype=float)).all():
0528                 finite_inference = False
0529                 break
0530         except (TypeError, ValueError):
0531             finite_inference = False
0532             break
0533     if not finite_inference:
0534         _append_unique(
0535             messages,
0536             "部分估计值或推断统计量不是有限数值, 通常表示矩阵求逆失败或数值溢出。",
0537         )
0538     diagnostics["converged"] = converged
0539     diagnostics["valid_inference"] = bool(
0540         diagnostics["valid_inference"] and converged and finite_inference
0541     )
0542     diagnostics["warnings"] = messages
0543     return diagnostics
0544 # Source file: backend/src/backend/analysis.py
0545 import math
0546 import warnings
0547 from typing import Literal
0548 import numpy as np
0549 import pandas as pd
0550 import statsmodels.api as sm

```

```

0551 from scipy import stats
0552 from backend.diagnostics import build_design_diagnostics, finalize_model_diagnostics
0553 from backend.quality import is_missing
0554 AnalysisMethod = Literal[
0555     "ols",
0556     "negative_binomial",
0557     "pearson",
0558     "spearman",
0559     "logistic",
0560 ]
0561 class AnalysisError(ValueError):
0562     pass
0563 def safe_float(value: object) -> float | None:
0564     converted = float(value)
0565     return converted if math.isfinite(converted) else None
0566 def prepare_numeric_data(
0567     dataframe: pd.DataFrame,
0568     columns: list[str],
0569 ) -> tuple[pd.DataFrame, int]:
0570     if len(columns) != len(set(columns)):
0571         raise AnalysisError("Each selected column must be unique.")
0572     numeric_data: dict[str, pd.Series] = {}
0573     for column in columns:
0574         if column not in dataframe.columns:
0575             raise AnalysisError(f"Column '{column}' does not exist.")
0576         original = dataframe[column]
0577         numeric = pd.to_numeric(original, errors="coerce")
0578         invalid_mask = ~original.map(is_missing) & numeric.isna()
0579         if invalid_mask.any():
0580             raise AnalysisError(f"Column '{column}' contains non-numeric values.")
0581         numeric_data[column] = numeric
0582     prepared = pd.DataFrame(numeric_data).dropna()
0583     return prepared, int(len(dataframe) - len(prepared))
0584 def run_correlation(
0585     dataframe: pd.DataFrame,
0586     *,
0587     method: AnalysisMethod,
0588     dependent_column: str,
0589     independent_columns: list[str],
0590 ) -> dict[str, object]:
0591     if len(independent_columns) != 1:
0592         raise AnalysisError("Pearson and Spearman require exactly two columns.")
0593     x_column = independent_columns[0]
0594     prepared, dropped_rows = prepare_numeric_data(
0595         dataframe, [dependent_column, x_column]
0596     )
0597     if len(prepared) < 3:
0598         raise AnalysisError("At least 3 complete observations are required.")
0599     if prepared[dependent_column].nunique() < 2 or prepared[x_column].nunique() < 2:
0600         raise AnalysisError("Correlation is undefined for a constant column.")

```

```

0601     if method == "pearson":
0602         result = stats.pearsonr(prepared[x_column], prepared[dependent_column])
0603     else:
0604         result = stats.spearmanr(prepared[x_column], prepared[dependent_column])
0605     return {
0606         "kind": "correlation",
0607         "method": method,
0608         "observations": len(prepared),
0609         "dropped_rows": dropped_rows,
0610         "dependent_column": dependent_column,
0611         "independent_columns": independent_columns,
0612         "correlation": {
0613             "coefficient": safe_float(result.statistic),
0614             "p_value": safe_float(result.pvalue),
0615         },
0616         "regression": None,
0617         "diagnostics": None,
0618     }
0619 def build_coefficients(
0620     results: object, method: AnalysisMethod
0621 ) -> list[dict[str, object]]:
0622     try:
0623         confidence_intervals = results.conf_int()
0624     except (ValueError, np.linalg.LinAlgError):
0625         confidence_intervals = pd.DataFrame(
0626             math.nan, index=results.params.index, columns=[0, 1]
0627         )
0628     coefficients = []
0629     for term in results.params.index:
0630         estimate = safe_float(results.params[term])
0631         effect_ratio = None
0632         if method in {"logistic", "negative_binomial"} and term != "alpha":
0633             try:
0634                 effect_ratio = safe_float(math.exp(float(results.params[term])))
0635             except OverflowError:
0636                 effect_ratio = None
0637         coefficients.append(
0638             {
0639                 "term": str(term),
0640                 "estimate": estimate,
0641                 "standard_error": safe_float(results.bse[term]),
0642                 "statistic": safe_float(results.tvalues[term]),
0643                 "p_value": safe_float(results.pvalues[term]),
0644                 "confidence_low": safe_float(confidence_intervals.loc[term, 0]),
0645                 "confidence_high": safe_float(confidence_intervals.loc[term, 1]),
0646                 "effect_ratio": effect_ratio,
0647             }
0648         )
0649     return coefficients
0650 def run_regression(

```

```

0651     dataframe: pd.DataFrame,
0652     *,
0653     method: AnalysisMethod,
0654     dependent_column: str,
0655     independent_columns: list[str],
0656 ) -> dict[str, object]:
0657     if not independent_columns:
0658         raise AnalysisError("Select at least one independent variable.")
0659     if dependent_column in independent_columns:
0660         raise AnalysisError(
0661             "The dependent variable cannot also be an independent variable."
0662         )
0663     prepared, dropped_rows = prepare_numeric_data(
0664         dataframe, [dependent_column, *independent_columns]
0665     )
0666     minimum_observations = len(independent_columns) + 3
0667     if len(prepared) < minimum_observations:
0668         raise AnalysisError(
0669             f"At least {minimum_observations} complete observations are required."
0670         )
0671     for column in independent_columns:
0672         if prepared[column].nunique() < 2:
0673             raise AnalysisError(f"Independent variable '{column}' is constant.")
0674     y = prepared[dependent_column]
0675     binary_mapping: dict[str, int] | None = None
0676     if method == "negative_binomial":
0677         if (y < 0).any() or not (y % 1 == 0).all():
0678             raise AnalysisError(
0679                 "Negative binomial regression requires non-negative integer counts."
0680             )
0681     elif method == "logistic":
0682         unique_values = sorted(y.unique().tolist())
0683         if len(unique_values) != 2:
0684             raise AnalysisError(
0685                 "Logistic regression requires exactly two outcome values."
0686             )
0687         binary_mapping = {str(unique_values[0]): 0, str(unique_values[1]): 1}
0688         y = y.map({unique_values[0]: 0, unique_values[1]: 1})
0689     x = sm.add_constant(prepared[independent_columns], has_constant="add")
0690     diagnostics = build_design_diagnostics(prepared, independent_columns)
0691     try:
0692         with warnings.catch_warnings(record=True) as captured:
0693             warnings.simplefilter("always")
0694             if method == "ols":
0695                 results = sm.OLS(y, x).fit()
0696             elif method == "negative_binomial":
0697                 results = sm.NegativeBinomial(y, x).fit(disp=False, maxiter=200)
0698             else:
0699                 results = sm.Logit(y, x).fit(disp=False, maxiter=200)
0700     except Exception as exc:

```

```

0701         raise AnalysisError(
0702             "The model could not converge. Check sample size, variable variation, and collinearity."
0703         ) from exc
0704     mle_results = getattr(results, "mle_retvals", None)
0705     converged = bool(mle_results.get("converged", True)) if mle_results else True
0706     diagnostics = finalize_model_diagnostics(
0707         diagnostics,
0708         converged=converged,
0709         inference_arrays=(results.params, results.bse, results.pvalues),
0710         captured_warnings=(str(item.message) for item in captured),
0711     )
0712     metrics = {
0713         "r_squared": safe_float(getattr(results, "rsquared", math.nan)),
0714         "adjusted_r_squared": safe_float(getattr(results, "rsquared_adj", math.nan)),
0715         "pseudo_r_squared": safe_float(getattr(results, "prsquared", math.nan)),
0716         "aic": safe_float(getattr(results, "aic", math.nan)),
0717         "bic": safe_float(getattr(results, "bic", math.nan)),
0718         "log_likelihood": safe_float(getattr(results, "llf", math.nan)),
0719     }
0720     return {
0721         "kind": "regression",
0722         "method": method,
0723         "observations": len(prepared),
0724         "dropped_rows": dropped_rows,
0725         "dependent_column": dependent_column,
0726         "independent_columns": independent_columns,
0727         "correlation": None,
0728         "regression": {
0729             "coefficients": build_coefficients(results, method),
0730             "metrics": metrics,
0731             "binary_mapping": binary_mapping,
0732         },
0733         "diagnostics": diagnostics,
0734     }
0735 def run_analysis(
0736     dataframe: pd.DataFrame,
0737     *,
0738     method: AnalysisMethod,
0739     dependent_column: str,
0740     independent_columns: list[str],
0741 ) -> dict[str, object]:
0742     if method in {"pearson", "spearman"}:
0743         return run_correlation(
0744             dataframe,
0745             method=method,
0746             dependent_column=dependent_column,
0747             independent_columns=independent_columns,
0748         )
0749     return run_regression(
0750         dataframe,

```

```

0751         method=method,
0752         dependent_column=dependent_column,
0753         independent_columns=independent_columns,
0754     )
0755 # Source file: backend/src/backend/spatial.py
0756 import math
0757 import warnings
0758 from typing import Literal
0759 import numpy as np
0760 import pandas as pd
0761 from esda.moran import Moran
0762 from libpysal.weights import KNN
0763 from mgwr.gwr import GWR
0764 from mgwr.sel_bw import Sel_BW
0765 from scipy import stats
0766 from scipy.sparse import SparseEfficiencyWarning
0767 from spreg import OLS, ML_Error, ML_Lag
0768 from backend.analysis import safe_float
0769 from backend.diagnostics import build_design_diagnostics, finalize_model_diagnostics
0770 from backend.quality import is_missing
0771 SpatialMethod = Literal["moran", "slm", "sem", "sdm", "gwr"]
0772 CoordinateType = Literal["geographic", "projected"]
0773 class SpatialAnalysisError(ValueError):
0774     pass
0775 MORAN_RANDOM_SEED = 20260826
0776 def _prepare_numeric(
0777     dataframe: pd.DataFrame, columns: list[str]
0778 ) -> tuple[pd.DataFrame, int]:
0779     if len(columns) != len(set(columns)):
0780         raise SpatialAnalysisError("Each selected column must be unique.")
0781     converted: dict[str, pd.Series] = {}
0782     for column in columns:
0783         if column not in dataframe.columns:
0784             raise SpatialAnalysisError(f"Column '{column}' does not exist.")
0785         original = dataframe[column]
0786         numeric = pd.to_numeric(original, errors="coerce")
0787         invalid = ~original.map(is_missing) & numeric.isna()
0788         if invalid.any():
0789             raise SpatialAnalysisError(
0790                 f"Column '{column}' contains non-numeric values."
0791             )
0792         converted[column] = numeric
0793     prepared = pd.DataFrame(converted, index=dataframe.index).dropna()
0794     prepared.insert(0, "__source_row__", prepared.index.to_numpy() + 2)
0795     return prepared.reset_index(drop=True), int(len(dataframe) - len(prepared))
0796 def _build_weights(
0797     coordinates: np.ndarray, coordinate_type: CoordinateType, neighbors: int
0798 ):
0799     count = len(coordinates)
0800     if not 1 <= neighbors < count:

```

```

0801         raise SpatialAnalysisError(
0802             f"The neighbor count must be between 1 and {count - 1}."
0803         )
0804     if coordinate_type == "geographic":
0805         longitude = coordinates[:, 0]
0806         latitude = coordinates[:, 1]
0807         if ((longitude < -180) | (longitude > 180)).any():
0808             raise SpatialAnalysisError("Longitude values must be between -180 and 180.")
0809         if ((latitude < -90) | (latitude > 90)).any():
0810             raise SpatialAnalysisError("Latitude values must be between -90 and 90.")
0811         lon_radians = np.radians(longitude)
0812         lat_radians = np.radians(latitude)
0813         # Chord distance on a unit sphere preserves great-circle nearest-neighbor order.
0814         weight_coordinates = np.column_stack(
0815             (
0816                 np.cos(lat_radians) * np.cos(lon_radians),
0817                 np.cos(lat_radians) * np.sin(lon_radians),
0818                 np.sin(lat_radians),
0819             )
0820         )
0821     else:
0822         weight_coordinates = coordinates
0823     weights = KNN.from_array(weight_coordinates, k=neighbors)
0824     weights.transform = "r"
0825     return weights
0826 def _moran_result(values: np.ndarray, weights: object) -> dict[str, object]:
0827     random_state = np.random.get_state()
0828     try:
0829         np.random.seed(MORAN_RANDOM_SEED)
0830         result = Moran(values, weights, permutations=999, two_tailed=True)
0831     finally:
0832         np.random.set_state(random_state)
0833     return {
0834         "i": safe_float(result.I),
0835         "expected_i": safe_float(result.EI),
0836         "z_score": safe_float(result.z_norm),
0837         "p_normal": safe_float(result.p_norm),
0838         "p_permutation": safe_float(result.p_sim),
0839         "permutations": 999,
0840         "random_seed": MORAN_RANDOM_SEED,
0841     }
0842 def _model_selection_diagnostics(
0843     prepared: pd.DataFrame,
0844     dependent_column: str,
0845     independent_columns: list[str],
0846     weights: object,
0847 ) -> dict[str, object]:
0848     y = prepared[[dependent_column]].to_numpy(dtype=float)
0849     x = prepared[independent_columns].to_numpy(dtype=float)
0850     try:

```

```

0851         with warnings.catch_warnings(record=True) as captured:
0852             warnings.simplefilter("always")
0853             baseline = OLS(
0854                 y,
0855                 x,
0856                 w=weights,
0857                 spat_diag=True,
0858                 moran=True,
0859                 name_y=dependent_column,
0860                 name_x=independent_columns,
0861             )
0862     except (
0863         FloatingPointError,
0864         RuntimeError,
0865         TypeError,
0866         ValueError,
0867         ZeroDivisionError,
0868         np.linalg.LinAlgError,
0869     ) as exc:
0870         return {
0871             "available": False,
0872             "baseline_residual_moran": None,
0873             "tests": [],
0874             "recommendation": "基础 OLS 空间诊断计算失败，请先处理共线性或数值尺度问题。",
0875             "warnings": [str(exc)],
0876         }
0877     test_definitions = [
0878         ("LM-Error", baseline.lm_error),
0879         ("LM-Lag", baseline.lm_lag),
0880         ("Robust LM-Error", baseline.rlm_error),
0881         ("Robust LM-Lag", baseline.rlm_lag),
0882         ("LM-SARMA", baseline.lm_sarma),
0883     ]
0884     tests = [
0885         {
0886             "name": name,
0887             "statistic": safe_float(result[0]),
0888             "p_value": safe_float(result[1]),
0889         }
0890         for name, result in test_definitions
0891     ]
0892     robust_error_p = float(baseline.rlm_error[1])
0893     robust_lag_p = float(baseline.rlm_lag[1])
0894     if robust_lag_p < 0.05 <= robust_error_p:
0895         recommendation = "稳健 LM-Lag 显著而稳健 LM-Error 不显著，优先考虑 SLM/SAR。"
0896     elif robust_error_p < 0.05 <= robust_lag_p:
0897         recommendation = "稳健 LM-Error 显著而稳健 LM-Lag 不显著，优先考虑 SEM。"
0898     elif robust_lag_p < 0.05 and robust_error_p < 0.05:
0899         recommendation = "两项稳健检验均显著，可能同时存在滞后和误差依赖；建议比较 SDM 等更完整设定。"
0900     else:

```

```

0901         recommendation = "两项稳健检验均不显著, 当前证据不足以支持 SLM 或 SEM; 可先保留 OLS 并结合理论判断。"
0902     moran_i, moran_z, moran_p = baseline.moran_res
0903     return {
0904         "available": True,
0905         "baseline_residual_moran": {
0906             "i": safe_float(moran_i),
0907             "z_score": safe_float(moran_z),
0908             "p_value": safe_float(moran_p),
0909         },
0910         "tests": tests,
0911         "recommendation": recommendation,
0912         "warnings": list(dict.fromkeys(str(item.message) for item in captured)),
0913     }
0914 def _spatial_impacts(
0915     model: object, method: SpatialMethod, independent_columns: list[str]
0916 ) -> list[dict[str, object]]:
0917     if method not in ["slm", "sdm"]:
0918         return []
0919     estimates = {
0920         name: float(value)
0921         for name, value in zip(model.name_x, np.asarray(model.betas).reshape(-1))
0922     }
0923     rho = float(model.rho)
0924     denominator = 1 - rho
0925     if abs(denominator) < 1e-12:
0926         return [
0927             {"term": column, "direct": None, "indirect": None, "total": None}
0928             for column in independent_columns
0929         ]
0930     impacts = []
0931     for column in independent_columns:
0932         direct = estimates[column]
0933         lagged_x = estimates.get(f"W_{column}", 0.0) if method == "sdm" else 0.0
0934         total = (direct + lagged_x) / denominator
0935         impacts.append(
0936             {
0937                 "term": column,
0938                 "direct": safe_float(direct),
0939                 "indirect": safe_float(total - direct),
0940                 "total": safe_float(total),
0941             }
0942         )
0943     return impacts
0944 def _coefficient_rows(model: object) -> list[dict[str, object]]:
0945     names = list(model.name_x)
0946     estimates = np.asarray(model.betas).reshape(-1)
0947     standard_errors = np.asarray(model.std_err).reshape(-1)
0948     statistics = list(model.z_stat)
0949     rows = []
0950     for index, name in enumerate(names):

```

```

0951     estimate = safe_float(estimates[index])
0952     standard_error = safe_float(standard_errors[index])
0953     statistic, p_value = statistics[index]
0954     confidence_low = (
0955         estimate - 1.96 * standard_error
0956         if estimate is not None and standard_error is not None
0957         else None
0958     )
0959     confidence_high = (
0960         estimate + 1.96 * standard_error
0961         if estimate is not None and standard_error is not None
0962         else None
0963     )
0964     rows.append(
0965         {
0966             "term": str(name),
0967             "estimate": estimate,
0968             "standard_error": standard_error,
0969             "statistic": safe_float(statistic),
0970             "p_value": safe_float(p_value),
0971             "confidence_low": confidence_low,
0972             "confidence_high": confidence_high,
0973         }
0974     )
0975     return rows
0976 def _run_moran(
0977     prepared: pd.DataFrame,
0978     variable: str,
0979     weights: object,
0980     diagnostics: dict[str, object],
0981 ) -> dict[str, object]:
0982     values = prepared[variable].to_numpy(dtype=float)
0983     if np.unique(values).size < 2:
0984         raise SpatialAnalysisError("Moran's I is undefined for a constant variable.")
0985     return {
0986         "kind": "moran",
0987         "moran": _moran_result(values, weights),
0988         "regression": None,
0989         "gwr": None,
0990         "residual_moran": None,
0991         "model_selection": None,
0992         "diagnostics": diagnostics,
0993     }
0994 def _run_spatial_regression(
0995     prepared: pd.DataFrame,
0996     method: SpatialMethod,
0997     dependent_column: str,
0998     independent_columns: list[str],
0999     weights: object,
1000     diagnostics: dict[str, object],

```

```

1001 ) -> dict[str, object]:
1002     y = prepared[[dependent_column]].to_numpy(dtype=float)
1003     x = prepared[independent_columns].to_numpy(dtype=float)
1004     try:
1005         with warnings.catch_warnings(record=True) as captured:
1006             warnings.simplefilter("always")
1007             warnings.filterwarnings("ignore", category=SparseEfficiencyWarning)
1008             if method == "sem":
1009                 model = ML_Error(
1010                     y,
1011                     x,
1012                     weights,
1013                     method="LU",
1014                     name_y=dependent_column,
1015                     name_x=independent_columns,
1016                 )
1017             else:
1018                 model = ML_Lag(
1019                     y,
1020                     x,
1021                     weights,
1022                     slx_lags=1 if method == "sdm" else 0,
1023                     method="LU",
1024                     spat_impacts="simple",
1025                     name_y=dependent_column,
1026                     name_x=independent_columns,
1027                 )
1028     except Exception as exc:
1029         raise SpatialAnalysisError(
1030             "The spatial model could not be estimated. Check variable scaling, "
1031             "collinearity, sample size, and the coordinate configuration."
1032         ) from exc
1033     diagnostics = finalize_model_diagnostics(
1034         diagnostics,
1035         converged=True,
1036         inference_arrays=(
1037             model.betas,
1038             model.std_err,
1039             [item[1] for item in model.z_stat],
1040         ),
1041         captured_warnings=(str(item.message) for item in captured),
1042         model_warning=(str(model.warning) if getattr(model, "warning", None) else None),
1043     )
1044     return {
1045         "kind": "spatial_regression",
1046         "moran": None,
1047         "gwr": None,
1048         "regression": {
1049             "coefficients": _coefficient_rows(model),
1050             "spatial_impacts": _spatial_impacts(model, method, independent_columns),

```

```

1051         "impact_method": "simple",
1052         "metrics": {
1053             "pseudo_r_squared": safe_float(model.pr2),
1054             "aic": safe_float(model.aic),
1055             "bic": safe_float(model.schwarz),
1056             "log_likelihood": safe_float(model.logll),
1057             "rho": safe_float(getattr(model, "rho", math.nan)),
1058             "lambda": safe_float(getattr(model, "lam", math.nan)),
1059         },
1060     },
1061     "residual_moran": _moran_result(np.asarray(model.u).reshape(-1), weights),
1062     "diagnostics": diagnostics,
1063 }
1064 def _run_gwr(
1065     prepared: pd.DataFrame,
1066     dependent_column: str,
1067     independent_columns: list[str],
1068     x_column: str,
1069     y_column: str,
1070     coordinate_type: CoordinateType,
1071     diagnostics: dict[str, object],
1072     local_result_limit: int | None,
1073     weights: object,
1074 ) -> dict[str, object]:
1075     if len(prepared) > 5000:
1076         raise SpatialAnalysisError(
1077             "GWR is limited to 5,000 complete observations in this version."
1078         )
1079     coordinates = prepared[[x_column, y_column]].to_numpy(dtype=float)
1080     y = prepared[[dependent_column]].to_numpy(dtype=float)
1081     x = prepared[independent_columns].to_numpy(dtype=float)
1082     minimum_bandwidth = max(2 * (len(independent_columns) + 1) + 1, 5)
1083     if len(prepared) <= minimum_bandwidth:
1084         raise SpatialAnalysisError(
1085             f"GWR requires more than {minimum_bandwidth} complete observations."
1086         )
1087     try:
1088         with warnings.catch_warnings(record=True) as captured:
1089             warnings.simplefilter("always")
1090             selector = Sel_BW(
1091                 coordinates,
1092                 y,
1093                 x,
1094                 fixed=False,
1095                 spherical=coordinate_type == "geographic",
1096                 n_jobs=1,
1097             )
1098             bandwidth = selector.search(bw_min=minimum_bandwidth)
1099             model = GWR(
1100                 coordinates,

```

```

1101         y,
1102         x,
1103         bandwidth,
1104         fixed=False,
1105         spherical=coordinate_type == "geographic",
1106         n_jobs=1,
1107     ).fit()
1108 except Exception as exc:
1109     raise SpatialAnalysisError(
1110         "GWR could not be estimated. Check duplicate coordinates, variable "
1111         "scaling, collinearity, and sample size."
1112     ) from exc
1113 diagnostics = finalize_model_diagnostics(
1114     diagnostics,
1115     converged=True,
1116     inference_arrays=(model.params, model.bse, model.tvalues),
1117     captured_warnings=(str(item.message) for item in captured),
1118 )
1119 term_names = ["CONSTANT", *independent_columns]
1120 local_rows = []
1121 local_count = (
1122     len(prepared)
1123     if local_result_limit is None
1124     else min(len(prepared), local_result_limit)
1125 )
1126 for index in range(local_count):
1127     row = {
1128         "source_row": int(prepared.iloc[index]["_source_row"]),
1129         "x_column": safe_float(coordinates[index, 0]),
1130         "y_column": safe_float(coordinates[index, 1]),
1131         "observed": safe_float(y[index, 0]),
1132         "predicted": safe_float(model.preds[index, 0]),
1133         "residual": safe_float(model.resid_response[index]),
1134         "local_r_squared": safe_float(model.localR2[index, 0]),
1135     }
1136     for term_index, term in enumerate(term_names):
1137         row[f"coefficient_{term}"] = safe_float(model.params[index, term_index])
1138         row[f"standard_error_{term}"] = safe_float(model.bse[index, term_index])
1139         row[f"t_value_{term}"] = safe_float(model.tvalues[index, term_index])
1140         row[f"p_value_unadjusted_{term}"] = safe_float(
1141             2 * stats.norm.sf(abs(model.tvalues[index, term_index]))
1142         )
1143     local_rows.append(row)
1144 summaries = []
1145 for index, term in enumerate(term_names):
1146     values = model.params[:, index]
1147     summaries.append(
1148         {
1149             "term": term,
1150             "mean": safe_float(np.mean(values)),

```

```

1151         "standard_deviation": safe_float(np.std(values, ddof=0)),
1152         "minimum": safe_float(np.min(values)),
1153         "median": safe_float(np.median(values)),
1154         "maximum": safe_float(np.max(values)),
1155     }
1156 )
1157 return {
1158     "kind": "gwr",
1159     "moran": None,
1160     "regression": None,
1161     "gwr": {
1162         "bandwidth": safe_float(bandwidth),
1163         "metrics": {
1164             "r_squared": safe_float(model.R2),
1165             "adjusted_r_squared": safe_float(model.adj_R2),
1166             "aic": safe_float(model.aic),
1167             "aicc": safe_float(model.aicc),
1168         },
1169         "coefficient_summaries": summaries,
1170         "local_result_count": len(prepared),
1171         "local_preview": local_rows,
1172     },
1173     "residual_moran": _moran_result(
1174         np.asarray(model.resid_response).reshape(-1), weights
1175     ),
1176     "diagnostics": diagnostics,
1177 }
1178 def run_spatial_analysis(
1179     dataframe: pd.DataFrame,
1180     *,
1181     method: SpatialMethod,
1182     coordinate_type: CoordinateType,
1183     x_column: str,
1184     y_column: str,
1185     dependent_column: str,
1186     independent_columns: list[str],
1187     neighbors: int,
1188     include_full_local_results: bool = False,
1189 ) -> dict[str, object]:
1190     if x_column == y_column:
1191         raise SpatialAnalysisError(
1192             "The X/longitude and Y/latitude columns must differ."
1193         )
1194     if method != "moran" and not independent_columns:
1195         raise SpatialAnalysisError("Select at least one independent variable.")
1196     if dependent_column in independent_columns:
1197         raise SpatialAnalysisError(
1198             "The dependent variable cannot also be an independent variable."
1199         )
1200     selected_columns = list(

```

```

1201         dict.fromkeys([x_column, y_column, dependent_column, *independent_columns])
1202     )
1203     prepared, dropped_rows = _prepare_numeric(dataframe, selected_columns)
1204     minimum = max(neighbors + 1, len(independent_columns) + 4, 4)
1205     if len(prepared) < minimum:
1206         raise SpatialAnalysisError(
1207             f"At least {minimum} complete observations are required."
1208         )
1209     if not np.isfinite(prepared[selected_columns].to_numpy(dtype=float)).all():
1210         raise SpatialAnalysisError("Selected columns contain infinite values.")
1211     if prepared[[x_column, y_column]].drop_duplicates().shape[0] < neighbors + 1:
1212         raise SpatialAnalysisError("There are too few distinct coordinate pairs.")
1213     for column in independent_columns:
1214         if prepared[column].nunique() < 2:
1215             raise SpatialAnalysisError(f"Independent variable '{column}' is constant.")
1216     coordinate_duplicates = int(
1217         prepared.duplicated(subset=[x_column, y_column], keep=False).sum()
1218     )
1219     coordinates = prepared[[x_column, y_column]].to_numpy(dtype=float)
1220     weights = _build_weights(coordinates, coordinate_type, neighbors)
1221     if method == "moran":
1222         diagnostics: dict[str, object] = {
1223             "converged": True,
1224             "valid_inference": True,
1225             "rank": None,
1226             "parameter_count": None,
1227             "condition_number": None,
1228             "raw_condition_number": None,
1229             "scale_ratio": None,
1230             "max_vif": None,
1231             "vif": [],
1232             "warnings": [],
1233         }
1234     else:
1235         diagnostics = build_design_diagnostics(prepared, independent_columns)
1236         diagnostic_warnings = diagnostics["warnings"]
1237     if coordinate_duplicates:
1238         diagnostic_warnings.append(
1239             f"有 {coordinate_duplicates} 行使用重复坐标；KNN/GWR 结果可能受重合点影响。"
1240         )
1241     if weights.n_components > 1:
1242         diagnostic_warnings.append(
1243             f"空间权重包含 {weights.n_components} 个互不连通的分量，建议检查坐标或增大 K 值。"
1244         )
1245     if method == "moran":
1246         specific = _run_moran(prepared, dependent_column, weights, diagnostics)
1247     elif method == "gwr":
1248         specific = _run_gwr(
1249             prepared,
1250             dependent_column,

```

```

1251         independent_columns,
1252         x_column,
1253         y_column,
1254         coordinate_type,
1255         diagnostics,
1256         None if include_full_local_results else 100,
1257         weights,
1258     )
1259     else:
1260         specific = _run_spatial_regression(
1261             prepared,
1262             method,
1263             dependent_column,
1264             independent_columns,
1265             weights,
1266             diagnostics,
1267         )
1268     if method != "moran":
1269         specific["model_selection"] = _model_selection_diagnostics(
1270             prepared,
1271             dependent_column,
1272             independent_columns,
1273             weights,
1274         )
1275     return {
1276         "method": method,
1277         "observations": len(prepared),
1278         "dropped_rows": dropped_rows,
1279         "coordinate_type": coordinate_type,
1280         "x_column": x_column,
1281         "y_column": y_column,
1282         "dependent_column": dependent_column,
1283         "independent_columns": independent_columns,
1284         "weights": {
1285             "type": "knn",
1286             "neighbors": neighbors,
1287             "transformation": "row_standardized",
1288             "components": int(weights.n_components),
1289         },
1290         **specific,
1291     }
1292 # Source file: backend/src/backend/reporting.py
1293 from io import BytesIO
1294 import pandas as pd
1295 def analysis_tables(result: dict[str, object]) -> tuple[pd.DataFrame, pd.DataFrame]:
1296     summary_rows = [
1297         {"item": "method", "value": result["method"]},
1298         {"item": "observations", "value": result["observations"]},
1299         {"item": "dropped_rows", "value": result["dropped_rows"]},
1300         {"item": "dependent_column", "value": result["dependent_column"]},

```

```

1301     {
1302         "item": "independent_columns",
1303         "value": ", ".join(result["independent_columns"]),
1304     },
1305 ]
1306 correlation = result.get("correlation")
1307 if correlation:
1308     summary_rows.extend(
1309         {"item": key, "value": value} for key, value in correlation.items()
1310     )
1311     detail = pd.DataFrame([correlation])
1312 else:
1313     regression = result["regression"]
1314     summary_rows.extend(
1315         {"item": key, "value": value}
1316         for key, value in regression["metrics"].items()
1317     )
1318     detail = pd.DataFrame(regression["coefficients"])
1319 diagnostics = result.get("diagnostics")
1320 if diagnostics:
1321     summary_rows.extend(
1322         {"item": key, "value": diagnostics[key]}
1323         for key in (
1324             "converged",
1325             "valid_inference",
1326             "rank",
1327             "parameter_count",
1328             "condition_number",
1329             "raw_condition_number",
1330             "scale_ratio",
1331             "max_vif",
1332         )
1333     )
1334     summary_rows.extend(
1335         {"item": f"warning_{index}", "value": message}
1336         for index, message in enumerate(diagnostics["warnings"], start=1)
1337     )
1338     return pd.DataFrame(summary_rows), detail
1339 def export_analysis_csv(result: dict[str, object]) -> bytes:
1340     _, detail = analysis_tables(result)
1341     return detail.to_csv(index=False, lineterminator="\n").encode("utf-8-sig")
1342 def export_analysis_xlsx(result: dict[str, object]) -> bytes:
1343     summary, detail = analysis_tables(result)
1344     output = BytesIO()
1345     with pd.ExcelWriter(output, engine="openpyxl") as writer:
1346         summary.to_excel(writer, sheet_name="summary", index=False)
1347         detail.to_excel(writer, sheet_name="coefficients", index=False)
1348         diagnostics = result.get("diagnostics")
1349         if diagnostics and diagnostics["vif"]:
1350             pd.DataFrame(diagnostics["vif"]).to_excel(

```

```

1351         writer, sheet_name="vif", index=False
1352     )
1353     return output.getvalue()
1354 def spatial_tables(result: dict[str, object]) -> dict[str, pd.DataFrame]:
1355     weights = result["weights"]
1356     summary_rows = [
1357         {"item": "method", "value": result["method"]},
1358         {"item": "observations", "value": result["observations"]},
1359         {"item": "dropped_rows", "value": result["dropped_rows"]},
1360         {"item": "coordinate_type", "value": result["coordinate_type"]},
1361         {"item": "x_column", "value": result["x_column"]},
1362         {"item": "y_column", "value": result["y_column"]},
1363         {"item": "dependent_column", "value": result["dependent_column"]},
1364         {
1365             "item": "independent_columns",
1366             "value": ", ".join(result["independent_columns"]),
1367         },
1368         {"item": "weights_type", "value": weights["type"]},
1369         {"item": "neighbors", "value": weights["neighbors"]},
1370         {"item": "weights_transformation", "value": weights["transformation"]},
1371         {"item": "connected_components", "value": weights["components"]},
1372     ]
1373     tables: dict[str, pd.DataFrame] = {}
1374     if result.get("moran"):
1375         summary_rows.extend(
1376             {"item": key, "value": value} for key, value in result["moran"].items()
1377         )
1378         tables["moran"] = pd.DataFrame(
1379             [{"item": key, "value": value} for key, value in result["moran"].items()]
1380         )
1381     elif result.get("regression"):
1382         regression = result["regression"]
1383         summary_rows.extend(
1384             {"item": key, "value": value}
1385             for key, value in regression["metrics"].items()
1386         )
1387         tables["coefficients"] = pd.DataFrame(regression["coefficients"])
1388         if regression["spatial_impacts"]:
1389             tables["spatial_impacts"] = pd.DataFrame(regression["spatial_impacts"])
1390     else:
1391         gwr = result["gwr"]
1392         summary_rows.append({"item": "bandwidth", "value": gwr["bandwidth"]})
1393         summary_rows.extend(
1394             {"item": key, "value": value} for key, value in gwr["metrics"].items()
1395         )
1396         tables["coefficient_summary"] = pd.DataFrame(gwr["coefficient_summaries"])
1397         tables["local_results"] = pd.DataFrame(gwr["local_preview"])
1398     residual_moran = result.get("residual_moran")
1399     if residual_moran:
1400         summary_rows.extend(

```

```

1401         {"item": f"residual_moran_{key}", "value": value}
1402     for key, value in residual_moran.items()
1403 )
1404     tables["residual_moran"] = pd.DataFrame(
1405         [{"item": key, "value": value} for key, value in residual_moran.items()]
1406     )
1407     model_selection = result.get("model_selection")
1408     if model_selection:
1409         summary_rows.append(
1410             {
1411                 "item": "model_selection_recommendation",
1412                 "value": model_selection["recommendation"],
1413             }
1414         )
1415     baseline_moran = model_selection["baseline_residual_moran"]
1416     if baseline_moran:
1417         summary_rows.extend(
1418             {"item": f"baseline_residual_moran_{key}", "value": value}
1419             for key, value in baseline_moran.items()
1420         )
1421     selection_rows = list(model_selection["tests"])
1422     if selection_rows:
1423         selection_table = pd.DataFrame(selection_rows)
1424         selection_table["recommendation"] = model_selection["recommendation"]
1425         tables["model_selection"] = selection_table
1426     diagnostics = result["diagnostics"]
1427     diagnostic_rows = [
1428         {"item": key, "value": diagnostics[key]}
1429         for key in (
1430             "converged",
1431             "valid_inference",
1432             "rank",
1433             "parameter_count",
1434             "condition_number",
1435             "raw_condition_number",
1436             "scale_ratio",
1437             "max_vif",
1438         )
1439     ]
1440     diagnostic_rows.extend(
1441         {"item": f"warning_{index}", "value": message}
1442         for index, message in enumerate(diagnostics["warnings"], start=1)
1443     )
1444     tables["summary"] = pd.DataFrame(summary_rows)
1445     tables["diagnostics"] = pd.DataFrame(diagnostic_rows)
1446     if diagnostics["vif"]:
1447         tables["vif"] = pd.DataFrame(diagnostics["vif"])
1448     return tables
1449 def export_spatial_csv(result: dict[str, object]) -> bytes:
1450     tables = spatial_tables(result)

```

```

1451     if "local_results" in tables:
1452         selected = tables["local_results"]
1453     elif "coefficients" in tables:
1454         selected = tables["coefficients"]
1455     elif "moran" in tables:
1456         selected = tables["moran"]
1457     else:
1458         selected = tables["summary"]
1459     diagnostics = result["diagnostics"]
1460     weights = result["weights"]
1461     metadata = {
1462         "report_method": result["method"],
1463         "report_observations": result["observations"],
1464         "report_dropped_rows": result["dropped_rows"],
1465         "report_coordinate_type": result["coordinate_type"],
1466         "report_neighbors": weights["neighbors"],
1467         "report_converged": diagnostics["converged"],
1468         "report_valid_inference": diagnostics["valid_inference"],
1469         "report_condition_number": diagnostics["condition_number"],
1470         "report_raw_condition_number": diagnostics["raw_condition_number"],
1471         "report_max_vif": diagnostics["max_vif"],
1472         "report_warnings": " | ".join(diagnostics["warnings"]),
1473     }
1474     residual_moran = result.get("residual_moran")
1475     if residual_moran:
1476         metadata.update(
1477             {
1478                 "report_residual_moran_i": residual_moran["i"],
1479                 "report_residual_moran_p_permutation": residual_moran["p_permutation"],
1480             }
1481         )
1482     model_selection = result.get("model_selection")
1483     if model_selection:
1484         metadata["report_model_recommendation"] = model_selection["recommendation"]
1485     exported = selected.copy()
1486     for column, value in reversed(metadata.items()):
1487         exported.insert(0, column, value)
1488     return exported.to_csv(index=False, lineterminator="\n").encode("utf-8-sig")
1489 def export_spatial_xlsx(result: dict[str, object]) -> bytes:
1490     output = BytesIO()
1491     with pd.ExcelWriter(output, engine="openpyxl") as writer:
1492         for sheet_name, table in spatial_tables(result).items():
1493             table.to_excel(writer, sheet_name=sheet_name[:31], index=False)
1494     return output.getvalue()
1495 # Source file: backend/src/backend/main.py
1496 import json
1497 from io import BytesIO
1498 from pathlib import Path
1499 from typing import Annotated
1500 from urllib.parse import quote

```

```

2365   } finally {
2366       isCleaning.value = false
2367   }
2368 }

2369 async function downloadCleaningResult(outputFormat: ExportFormat, table: ExportTable = 'cleaned') {
2370     if (!selectedFile.value) return
2371     const target = `${outputFormat}-${table}`
2372     exportingTarget.value = target
2373     exportMessage.value = ''
2374     errorMessage.value = ''
2375     try {
2376         const filename = await exportCleaning(
2377             selectedFile.value,
2378             {
2379                 missingAction: missingAction.value,
2380                 trimWhitespace: trimWhitespace.value,
2381                 removeLineBreaks: removeLineBreaks.value,
2382                 standardizationMethod: standardizationMethod.value,
2383                 standardizationColumns: standardizationColumns.value,
2384             },
2385             outputFormat,
2386             table,
2387         )
2388         exportMessage.value = `已导出 ${filename}`
2389     } catch (error) {
2390         errorMessage.value = error instanceof Error ? error.message : '导出失败。'
2391     } finally {
2392         exportingTarget.value = ''
2393     }
2394 }

2395 function formatFileSize(size: number) {
2396     if (size < 1024) return `${size} B`
2397     if (size < 1024 * 1024) return `${(size / 1024).toFixed(1)} KB`
2398     return `${(size / 1024 / 1024).toFixed(1)} MB`
2399 }

2400 function formatRatio(ratio: number) {
2401     return `${(ratio * 100).toFixed(1)}%`
2402 }

2403 </script>
2404 <template>
2405     <main class="app-shell">
2406         <header class="hero">
2407             <div>
2408                 <p class="eyebrow">LOCAL DATA TOOLKIT</p>
2409                 <h1>DataEX-G</h1>
2410             </div>
2411             <div class="local-badge"><span></span> 本地处理</div>
2412         </header>
2413         <nav class="workspace-tabs" aria-label="工作区">
2414             <button

```

```

2415         :class="{ active: activeWorkspace === 'cleaning' }"
2416         @click="activeWorkspace = 'cleaning'"
2417     >
2418         数据清洗
2419     </button>
2420 <button
2421     :class="{ active: activeWorkspace === 'analysis' }"
2422     @click="activeWorkspace = 'analysis'"
2423 >
2424     回归分析方法
2425 </button>
2426 <button
2427     :class="{ active: activeWorkspace === 'spatial' }"
2428     @click="activeWorkspace = 'spatial'"
2429 >
2430     空间分析
2431 </button>
2432 </nav>
2433 <div v-show="activeWorkspace === 'cleaning'">
2434     <section class="panel upload-panel">
2435         <div class="section-heading">
2436             <div>
2437                 <p class="step-label">步骤 01</p>
2438                 <h2>选择数据文件</h2>
2439             </div>
2440             <p>支持 CSV、XLSX，单个文件不超过 20 MB</p>
2441         </div>
2442         <label class="drop-zone" @dragover.prevent @drop.prevent="onDrop">
2443             <input type="file" accept=".csv,.xlsx" @change="onFileSelected" />
2444             <span class="upload-icon">↑</span>
2445             <strong>点击选择或拖放文件到这里</strong>
2446             <small>文件只会发送到你电脑上的 Python 服务</small>
2447         </label>
2448         <div v-if="selectedFile" class="selected-file">
2449             <div>
2450                 <strong>{{ selectedFile.name }}</strong>
2451                 <span>{{ formatFileSize(selectedFile.size) }}</span>
2452             </div>
2453             <button :disabled="isLoading" @click="analyzeSelectedFile">
2454                 {{ isLoading ? '分析中...' : '开始分析' }}
2455             </button>
2456         </div>
2457         <p v-if="errorMessage" class="error-message">{{ errorMessage }}</p>
2458     </section>
2459 <template v-if="result">
2460     <section class="summary-grid" aria-label="文件摘要">
2461         <article class="metric-card">
2462             <span>数据行数</span>
2463             <strong>{{ result.row_count }}</strong>
2464         </article>

```

```

2465     <article class="metric-card">
2466         <span>字段数量</span>
2467         <strong>{{ result.column_count }}</strong>
2468     </article>
2469     <article class="metric-card" :class="{ warning: result.quality.missing_cell_count }">
2470         <span>空值单元格</span>
2471         <strong>{{ result.quality.missing_cell_count }}</strong>
2472     </article>
2473     <article class="metric-card" :class="{ warning: result.quality.duplicate_row_count }">
2474         <span>重复数据行</span>
2475         <strong>{{ result.quality.duplicate_row_count }}</strong>
2476     </article>
2477     <article class="metric-card" :class="{ warning: result.quality.whitespace_cell_count }">
2478         <span>前后空格</span>
2479         <strong>{{ result.quality.whitespace_cell_count }}</strong>
2480     </article>
2481     <article class="metric-card" :class="{ warning: result.quality.line_break_cell_count }">
2482         <span>回车/换行</span>
2483         <strong>{{ result.quality.line_break_cell_count }}</strong>
2484     </article>
2485 </section>
2486 <section class="panel">
2487     <div class="section-heading">
2488         <div>
2489             <p class="step-label">步骤 02</p>
2490             <h2>质量报告</h2>
2491         </div>
2492         <span class="result-badge" :class="{ clean: !hasIssues }">
2493             {{ hasIssues ? '发现需要关注的问题' : '未发现明显问题' }}
2494         </span>
2495     </div>
2496     <div class="quality-table-wrap">
2497         <table class="quality-table">
2498             <thead>
2499                 <tr>
2500                     <th>字段</th>
2501                     <th>pandas 类型</th>
2502                     <th>空值</th>
2503                     <th>检测到的内容类型</th>
2504                     <th>文本问题</th>
2505                     <th>状态</th>
2506                 </tr>
2507             </thead>
2508             <tbody>
2509                 <tr v-for="column in result.quality.columns" :key="column.name">
2510                     <td class="column-name">{{ column.name }}</td>
2511                     <td>
2512                         <code>{{ column.pandas_dtype }}</code>
2513                     </td>
2514                     <td>{{ column.missing_count }} ({{ formatRatio(column.missing_ratio) }})</td>

```

```

2515         <td>
2516             <span
2517                 v-for="detected in column.detected_types"
2518                 :key="detected.type"
2519                 class="type-chip"
2520                 :title="detected.examples.join('、')"
2521             >
2522                 {{ detected.type }} · {{ detected.count }}
2523             </span>
2524             <span v-if="!column.detected_types.length" class="muted">无有效值</span>
2525         </td>
2526     <td>
2527         <span v-if="column.whitespace_count" class="issue-chip">
2528             前后空格 {{ column.whitespace_count }}
2529         </span>
2530         <span v-if="column.line_break_count" class="issue-chip">
2531             回车/换行 {{ column.line_break_count }}
2532         </span>
2533         <span v-if="!column.whitespace_count && !column.line_break_count" class="muted">
2534             无
2535         </span>
2536     </td>
2537 </tr>
2538 </tbody>
2539 </table>
2540 </div>
2541 </section>
2542 <section class="panel cleaning-panel">
2543     <div class="section-heading">
2544         <div>
2545             <p class="step-label">步骤 03</p>
2546             <h2>选择清洗规则</h2>
2547         </div>
2548         <p>只生成预览，不会覆盖原文件</p>
2549     </div>
2550     <div class="rule-group">
2551         <h3>空值处理</h3>
2552         <div class="radio-grid">
2553             <label class="rule-option">
2554                 <input v-model="missingAction" type="radio" value="none" />
2555                 <span><strong>暂不处理</strong><small>保留所有空值</small></span>
2556             </label>
2557             <label class="rule-option">
2558                 <input v-model="missingAction" type="radio" value="drop_rows" />
2559                 <span><strong>删除空值行</strong><small>删除包含任意空值的整行</small></span>
2560             </label>

```

```

2565     <label class="rule-option">
2566         <input v-model="missingAction" type="radio" value="extract_rows" />
2567         <span><strong>摘出为空值表</strong><small>从主表移出并生成独立表</small></span>
2568     </label>
2569     <label class="rule-option">
2570         <input v-model="missingAction" type="radio" value="fill_zero" />
2571         <span><strong>使用 0 替换</strong><small>将所有空值填充为数值 0</small></span>
2572     </label>
2573 </div>
2574 </div>
2575 <div class="rule-group">
2576     <h3>文本处理</h3>
2577     <div class="checkbox-list">
2578         <label>
2579             <input v-model="trimWhitespace" type="checkbox" />
2580             清除文本前后空格
2581             <small>检测到 {{ result.quality.whitespace_cell_count }} 个单元格</small>
2582         </label>
2583         <label>
2584             <input v-model="removeLineBreaks" type="checkbox" />
2585             清除回车和换行符
2586             <small>检测到 {{ result.quality.line_break_cell_count }} 个单元格</small>
2587         </label>
2588     </div>
2589 </div>
2590 <div class="rule-group">
2591     <h3>数据标准化</h3>
2592     <div class="standardization-options">
2593         <label>
2594             <input v-model="standardizationMethod" type="radio" value="none" />
2595             暂不标准化
2596         </label>
2597         <label>
2598             <input v-model="standardizationMethod" type="radio" value="min_max" />
2599             Min-Max 标准化
2600             <small>将数值缩放至 0 - 1</small>
2601         </label>
2602         <label>
2603             <input v-model="standardizationMethod" type="radio" value="z_score" />
2604             Z-score 标准化
2605             <small>转换为均值 0、标准差 1</small>
2606         </label>
2607     </div>
2608     <div v-if="standardizationMethod !== 'none'" class="column-selector">
2609         <p>选择需要标准化的数值列: </p>
2610         <label v-for="column in numericColumns" :key="column">
2611             <input v-model="standardizationColumns" type="checkbox" :value="column" />
2612             {{ column }}
2613         </label>
2614         <p v-if="!numericColumns.length" class="validation-message">

```

```

2615         当前数据没有可安全标准化的纯数值列。
2616     </p>
2617     <p v-else-if="!standardizationColumns.length" class="validation-message">
2618         请至少选择一个数值列。
2619     </p>
2620 </div>
2621 </div>
2622 <button
2623     class="primary-action"
2624     :disabled="isCleaning || !canPreviewCleaning"
2625     @click="previewSelectedCleaning"
2626 >
2627     {{ isCleaning ? '正在生成...' : '生成清洗预览' }}
2628 </button>
2629 <div v-if="cleaningResult" class="cleaning-result">
2630     <div class="cleaning-summary">
2631         <span>原始 {{ cleaningResult.original_row_count }} 行</span>
2632         <span>清洗后 {{ cleaningResult.cleaned_row_count }} 行</span>
2633         <span v-if="cleaningResult.extracted_row_count">
2634             摘出 {{ cleaningResult.extracted_row_count }} 行
2635         </span>
2636         <span>修改文本 {{ cleaningResult.summary.text_changed_cell_count }} 格</span>
2637         <span v-if="cleaningResult.summary.standardization_method === 'min_max'">
2638             Min-Max: {{ cleaningResult.summary.standardized_columns.join(', ') }}
2639         </span>
2640         <span v-if="cleaningResult.summary.standardization_method === 'z_score'">
2641             Z-score: {{ cleaningResult.summary.standardized_columns.join(', ') }}
2642         </span>
2643     </div>
2644     <h3>清洗后主表预览</h3>
2645     <div class="data-table-wrap compact-table">
2646         <table class="data-table">
2647             <thead>
2648                 <tr>
2649                     <th class="row-number">#</th>
2650                     <th v-for="column in cleaningResult.columns" :key="column">{{ column }}</th>
2651                 </tr>
2652             </thead>
2653             <tbody>
2654                 <tr v-for="(row, rowIndex) in cleaningResult.cleaned_preview" :key="rowIndex">
2655                     <td class="row-number">{{ rowIndex + 1 }}</td>
2656                     <td v-for="column in cleaningResult.columns" :key="column">
2657                         <span v-if="row[column] === null" class="missing-value">空值</span>
2658                         <span v-else>{{ row[column] }}</span>
2659                     </td>
2660                 </tr>
2661             </tbody>
2662         </table>
2663     </div>
2664     <template v-if="cleaningResult.extracted_row_count">

```

```

2665 <h3 class="extracted-heading">空值数据表预览</h3>
2666 <p class="muted">
2667     来源数据行: {{ cleaningResult.summary.extracted_row_numbers.join(', ') }}
2668 </p>
2669 <div class="data-table-wrap compact-table extracted-table">
2670     <table class="data-table">
2671         <thead>
2672             <tr>
2673                 <th class="row-number">#</th>
2674                 <th v-for="column in cleaningResult.columns" :key="column">{{ column }}</th>
2675             </tr>
2676         </thead>
2677         <tbody>
2678             <tr v-for="(row, rowIndex) in cleaningResult.extracted_preview" :key="rowIndex">
2679                 <td class="row-number">{{ rowIndex + 1 }}</td>
2680                 <td v-for="column in cleaningResult.columns" :key="column">
2681                     <span v-if="row[column] === null" class="missing-value">空值</span>
2682                     <span v-else>{{ row[column] }}</span>
2683                 </td>
2684             </tr>
2685         </tbody>
2686     </table>
2687 </div>
2688 </template>
2689 <div class="export-panel">
2690     <div>
2691         <h3>导出清洗结果</h3>
2692         <p>新文件不会覆盖原文件，文件名自动添加 -dataex。</p>
2693     </div>
2694     <div class="export-actions">
2695         <button
2696             :disabled="!!exportingTarget"
2697             @click="downloadCleaningResult('csv', 'cleaned')"
2698         >
2699             {{ exportingTarget === 'csv-cleaned' ? '导出中...' : '导出主表 CSV' }}
2700         </button>
2701         <button
2702             v-if="cleaningResult.extracted_row_count"
2703             :disabled="!!exportingTarget"
2704             @click="downloadCleaningResult('csv', 'extracted')"
2705         >
2706             {{ exportingTarget === 'csv-extracted' ? '导出中...' : '导出空值表 CSV' }}
2707         </button>
2708         <button
2709             class="xlsx-button"
2710             :disabled="!!exportingTarget"
2711             @click="downloadCleaningResult('xlsx')"
2712         >
2713             {{ exportingTarget === 'xlsx-cleaned' ? '导出中...' : '导出 XLSX' }}
2714         </button>

```

```

2715         </div>
2716         <p v-if="exportMessage" class="export-message">{{ exportMessage }}</p>
2717     </div>
2718 </div>
2719 </section>
2720 <section class="panel">
2721     <div class="section-heading">
2722         <div>
2723             <p class="step-label">原始数据</p>
2724             <h2>原始数据预览</h2>
2725         </div>
2726         <p>显示前 {{ result.preview.length }} 行</p>
2727     </div>
2728     <div class="data-table-wrap">
2729         <table class="data-table">
2730             <thead>
2731                 <tr>
2732                     <th class="row-number">#</th>
2733                     <th v-for="column in result.columns" :key="column">{{ column }}</th>
2734                 </tr>
2735             </thead>
2736             <tbody>
2737                 <tr v-for="(row, rowIndex) in result.preview" :key="rowIndex">
2738                     <td class="row-number">{{ rowIndex + 1 }}</td>
2739                     <td v-for="column in result.columns" :key="column">
2740                         <span v-if="row[column] === null" class="missing-value">空值</span>
2741                         <span v-else>{{ row[column] }}</span>
2742                     </td>
2743                 </tr>
2744             </tbody>
2745         </table>
2746     </div>
2747 </section>
2748 </template>
2749 </div>
2750 <div v-show="activeWorkspace === 'analysis'" class="workspace-view">
2751     <AnalysisWorkspace />
2752 </div>
2753 <div v-show="activeWorkspace === 'spatial'" class="workspace-view">
2754     <SpatialWorkspace />
2755 </div>
2756 </main>
2757 </template>
2758 <!-- Source file: frontend/src/components/AnalysisWorkspace.vue -->
2759 <script setup lang="ts">
2760 import { computed, ref, watch } from 'vue'
2761 import { exportAnalysis, previewFile, runAnalysis } from '@/services/api'
2762 import type {
2763     AnalysisMethod,
2764     AnalysisResponse,

```

```

2765   ExportFormat,
2766   FilePreviewResponse,
2767 } from '@types/analysis'
2768 const methods: Array<{ value: AnalysisMethod; label: string; description: string }> = [
2769   { value: 'ols', label: 'OLS', description: '连续因变量的普通最小二乘回归' },
2770   { value: 'negative_binomial', label: '负二项回归', description: '非负整数计数型因变量' },
2771   { value: 'pearson', label: 'Pearson', description: '两个数值变量的线性相关' },
2772   { value: 'spearman', label: 'Spearman', description: '两个数值变量的单调相关' },
2773   { value: 'logistic', label: 'Logistic', description: '只有两类结果的二元回归' },
2774 ]
2775 const selectedFile = ref<File | null>(null)
2776 const dataset = ref<FilePreviewResponse | null>(null)
2777 const selectedMethod = ref<AnalysisMethod>('ols')
2778 const dependentColumn = ref('')
2779 const independentColumns = ref<string[]>([])
2780 const correlationColumn = ref('')
2781 const result = ref<AnalysisResponse | null>(null)
2782 const errorMessage = ref('')
2783 const isReading = ref(false)
2784 const isRunning = ref(false)
2785 const exportingFormat = ref<ExportFormat | ''>('')
2786 const exportMessage = ref('')
2787 const numericColumns = computed(() =>
2788   (dataset.value?.quality.columns ?? []))
2789   .filter(
2790     (column) =>
2791       !column.mixed_types &&
2792       column.detected_types.length === 1 &&
2793       column.detected_types[0]?.type === 'number',
2794   )
2795   .map((column) => column.name),
2796 )
2797 const isCorrelation = computed(() => ['pearson', 'spearman'].includes(selectedMethod.value))
2798 const availableIndependentColumns = computed(() =>
2799   numericColumns.value.filter((column) => column !== dependentColumn.value),
2800 )
2801 const canRun = computed(() => {
2802   if (!selectedFile.value || !dependentColumn.value) return false
2803   return isCorrelation.value
2804     ? Boolean(correlationColumn.value)
2805     : independentColumns.value.length > 0
2806 })
2807 watch([selectedMethod, dependentColumn], () => {
2808   result.value = null
2809   errorMessage.value = ''
2810   exportMessage.value = ''
2811   independentColumns.value = independentColumns.value.filter(
2812     (column) => column !== dependentColumn.value,
2813   )
2814   if (correlationColumn.value === dependentColumn.value) correlationColumn.value = ''

```

```

2815 })
2816 async function onFileSelected(event: Event) {
2817     const input = event.target as HTMLInputElement
2818     const file = input.files?.[0]
2819     if (!file) return
2820     selectedFile.value = file
2821     dataset.value = null
2822     result.value = null
2823     dependentColumn.value = ''
2824     independentColumns.value = []
2825     correlationColumn.value = ''
2826     errorMessage.value = ''
2827     isReading.value = true
2828     try {
2829         dataset.value = await previewFile(file)
2830     } catch (error) {
2831         selectedFile.value = null
2832         errorMessage.value = error instanceof Error ? error.message : '读取字段失败。'
2833     } finally {
2834         isReading.value = false
2835     }
2836 }
2837 async function executeAnalysis() {
2838     if (!selectedFile.value || !canRun.value) return
2839     isRunning.value = true
2840     result.value = null
2841     errorMessage.value = ''
2842     try {
2843         result.value = await runAnalysis(
2844             selectedFile.value,
2845             selectedMethod.value,
2846             dependentColumn.value,
2847             isCorrelation.value ? [correlationColumn.value] : independentColumns.value,
2848         )
2849     } catch (error) {
2850         errorMessage.value = error instanceof Error ? error.message : '分析失败。'
2851     } finally {
2852         isRunning.value = false
2853     }
2854 }
2855 async function downloadAnalysisResult(outputFormat: ExportFormat) {
2856     if (!selectedFile.value || !result.value) return
2857     exportingFormat.value = outputFormat
2858     errorMessage.value = ''
2859     exportMessage.value = ''
2860     try {
2861         const filename = await exportAnalysis(
2862             selectedFile.value,
2863             selectedMethod.value,
2864             dependentColumn.value,

```

```

2865     isCorrelation.value ? [correlationColumn.value] : independentColumns.value,
2866     outputFormat,
2867   )
2868   exportMessage.value = `已导出 ${filename}`
2869 } catch (error) {
2870   errorMessage.value = error instanceof Error ? error.message : '结果导出失败。'
2871 } finally {
2872   exportingFormat.value = ''
2873 }
2874 }
2875 function formatNumber(value: number | null) {
2876   if (value === null) return '—'
2877   if (value !== 0 && Math.abs(value) < 0.0001) return value.toExponential(3)
2878   return value.toFixed(6)
2879 }
2880 function correlationStrength(value: number | null) {
2881   if (value === null) return '无法判断'
2882   const magnitude = Math.abs(value)
2883   if (magnitude >= 0.7) return '强相关'
2884   if (magnitude >= 0.4) return '中等相关'
2885   return '弱相关'
2886 }
2887 </script>
2888 <template>
2889   <section class="panel analysis-intro">
2890     <div class="section-heading">
2891       <div>
2892         <p class="step-label">ANALYSIS</p>
2893         <h2>回归分析方法</h2>
2894       </div>
2895       <p>OLS、负二项、Logistic 回归，以及 Pearson、Spearman 相关性</p>
2896     </div>
2897     <label class="analysis-file-picker">
2898       <input type="file" accept=".csv,.xlsx" @change="onFileSelected" />
2899       <span>{{ isReading ? '正在读取字段...' : '选择分析数据' }}</span>
2900       <small>{{ selectedFile?.name ?? '建议载入清洗后导出的 -dataex 文件' }}</small>
2901     </label>
2902     <p v-if="errorMessage" class="error-message">{{ errorMessage }}</p>
2903   </section>
2904   <template v-if="dataset">
2905     <section class="panel">
2906       <div class="analysis-file-summary">
2907         <span>{{ dataset.filename }}</span>
2908         <span>{{ dataset.row_count }} 行</span>
2909         <span>{{ numericColumns.length }} 个可用数值列</span>
2910       </div>
2911       <h3 class="analysis-section-title">选择分析方法</h3>
2912       <div class="method-grid">
2913         <label v-for="method in methods" :key="method.value" class="method-card">
2914           <input v-model="selectedMethod" type="radio" :value="method.value" />

```

```

2915         <strong>{{ method.label }}</strong>
2916         <small>{{ method.description }}</small>
2917     </label>
2918 </div>
2919 <div class="variable-grid">
2920     <label>
2921         <span>{{ isCorrelation ? '变量 Y' : '因变量 Y' }}</span>
2922         <select v-model="dependentColumn">
2923             <option value="" disabled>请选择数值列</option>
2924             <option v-for="column in numericColumns" :key="column" :value="column">
2925                 {{ column }}
2926             </option>
2927         </select>
2928     </label>
2929     <label v-if="isCorrelation">
2930         <span>变量 X</span>
2931         <select v-model="correlationColumn">
2932             <option value="" disabled>请选择另一个数值列</option>
2933             <option v-for="column in availableIndependentColumns" :key="column" :value="column">
2934                 {{ column }}
2935             </option>
2936         </select>
2937     </label>
2938     <div v-else class="independent-selector">
2939         <span>自变量 X (可多选) </span>
2940         <div>
2941             <label v-for="column in availableIndependentColumns" :key="column">
2942                 <input v-model="independentColumns" type="checkbox" :value="column" />
2943                 {{ column }}
2944             </label>
2945         </div>
2946     </div>
2947 </div>
2948 <p v-if="!numericColumns.length" class="analysis-warning">
2949     当前文件没有可安全分析的纯数值列，请先清洗或转换数据类型。
2950 </p>
2951 <button class="primary-action" :disabled="isRunning || !canRun" @click="executeAnalysis">
2952     {{ isRunning ? '正在计算...' : '运行分析' }}
2953 </button>
2954 </section>
2955 <section v-if="result" class="panel analysis-result">
2956     <div class="section-heading">
2957         <div>
2958             <p class="step-label">RESULT</p>
2959             <h2>分析结果</h2>
2960         </div>
2961         <p>有效样本 {{ result.observations }}，排除缺失行 {{ result.dropped_rows }}</p>
2962     </div>
2963     <div
2964         v-if="result.diagnostics"

```

```

2965     class="diagnostics-panel"
2966     :class="{ invalid: !result.diagnostics.valid_inference }"
2967 >
2968     <div class="diagnostics-heading">
2969         <div>
2970             <strong>
2971                 {{ result.diagnostics.valid_inference ? '模型推断有效' : '模型推断不可靠' }}
2972             </strong>
2973             <small>
2974                 {{ result.diagnostics.converged ? '模型已收敛' : '模型未收敛' }}
2975             </small>
2976         </div>
2977         <div class="diagnostic-values">
2978             <span>标准化条件数 {{ formatNumber(result.diagnostics.condition_number) }}</span>
2979             <span>原始条件数 {{ formatNumber(result.diagnostics.raw_condition_number) }}</span>
2980             <span>最大 VIF {{ formatNumber(result.diagnostics.max_vif) }}</span>
2981             <span>尺度比 {{ formatNumber(result.diagnostics.scale_ratio) }}</span>
2982         </div>
2983     </div>
2984     <ul v-if="result.diagnostics.warnings.length">
2985         <li v-for="warning in result.diagnostics.warnings" :key="warning">{{ warning }}</li>
2986     </ul>
2987     <p v-else>未检测到收敛、非有限数值或明显共线性问题。</p>
2988     <details v-if="result.diagnostics.vif.length">
2989         <summary>查看各变量 VIF</summary>
2990         <div class="vif-list">
2991             <span v-for="item in result.diagnostics.vif" :key="item.column">
2992                 {{ item.column }}: {{ formatNumber(item.vif) }}
2993             </span>
2994         </div>
2995     </details>
2996 </div>
2997 <div v-if="result.correlation" class="correlation-result">
2998     <article>
2999         <span>相关系数</span>
3000         <strong>{{ formatNumber(result.correlation.coefficient) }}</strong>
3001         <small>{{ correlationStrength(result.correlation.coefficient) }}</small>
3002     </article>
3003     <article>
3004         <span>p 值</span>
3005         <strong>{{ formatNumber(result.correlation.p_value) }}</strong>
3006         <small>{{ (result.correlation.p_value ?? 1) < 0.05 ? '统计显著' : '未达到 0.05' }}</small>
3007     </article>
3008 </div>
3009 <template v-if="result.regression">
3010     <div class="model-metrics">
3011         <article v-if="result.regression.metrics.r_squared !== null">
3012             <span>R2</span><strong>{{ formatNumber(result.regression.metrics.r_squared) }}</strong>
3013         </article>
3014         <article v-if="result.regression.metrics.adjusted_r_squared !== null">

```

```

3015         <span>调整 R2</span>
3016         <strong>{{ formatNumber(result.regression.metrics.adjusted_r_squared) }}</strong>
3017     </article>
3018     <article v-if="result.regression.metrics.pseudo_r_squared !== null">
3019         <span>伪 R2</span>
3020         <strong>{{ formatNumber(result.regression.metrics.pseudo_r_squared) }}</strong>
3021     </article>
3022     <article>
3023         <span>AIC</span><strong>{{ formatNumber(result.regression.metrics.aic) }}</strong>
3024     </article>
3025     <article>
3026         <span>BIC</span><strong>{{ formatNumber(result.regression.metrics.bic) }}</strong>
3027     </article>
3028 </div>
3029 <p v-if="result.regression.binary_mapping" class="binary-mapping">
3030     Logistic 编码:
3031     <span v-for="(value, label) in result.regression.binary_mapping" :key="label">
3032         {{ label }} → {{ value }}
3033     </span>
3034 </p>
3035 <div class="quality-table-wrap">
3036     <table class="quality-table">
3037         <thead>
3038             <tr>
3039                 <th>变量</th>
3040                 <th>系数</th>
3041                 <th>标准误</th>
3042                 <th>统计量</th>
3043                 <th>p 值</th>
3044                 <th>95% 置信区间</th>
3045                 <th v-if="selectedMethod === 'logistic'">优势比</th>
3046                 <th v-if="selectedMethod === 'negative_binomial'">发生率比</th>
3047             </tr>
3048         </thead>
3049         <tbody>
3050             <tr v-for="coefficient in result.regression.coefficients" :key="coefficient.term">
3051                 <td class="column-name">{{ coefficient.term }}</td>
3052                 <td>{{ formatNumber(coefficient.estimate) }}</td>
3053                 <td>{{ formatNumber(coefficient.standard_error) }}</td>
3054                 <td>{{ formatNumber(coefficient.statistic) }}</td>
3055                 <td>{{ formatNumber(coefficient.p_value) }}</td>
3056                 <td>
3057                     {{ formatNumber(coefficient.confidence_low) }} ~
3058                     {{ formatNumber(coefficient.confidence_high) }}
3059                 </td>
3060                 <td v-if="['logistic', 'negative_binomial'].includes(selectedMethod)">
3061                     {{ formatNumber(coefficient.effect_ratio) }}
3062                 </td>
3063             </tr>
3064         </tbody>

```

```

3065         </table>
3066     </div>
3067 </template>
3068 <p class="analysis-disclaimer">
3069     统计显著性和相关性不代表因果关系；正式结论还需检查模型假设、样本设计与业务背景。
3070 </p>
3071 <div class="export-panel analysis-export-panel">
3072     <div>
3073         <h3>导出分析结果</h3>
3074         <p>文件名自动添加 -analysis-dataex; XLSX 包含概览和系数工作表。</p>
3075     </div>
3076     <div class="export-actions">
3077         <button :disabled="!!exportingFormat" @click="downloadAnalysisResult('csv')">
3078             {{ exportingFormat === 'csv' ? '导出中...' : '导出 CSV' }}
3079         </button>
3080         <button
3081             class="xlsx-button"
3082             :disabled="!!exportingFormat"
3083             @click="downloadAnalysisResult('xlsx')">
3084             >
3085             {{ exportingFormat === 'xlsx' ? '导出中...' : '导出 XLSX' }}
3086         </button>
3087     </div>
3088     <p v-if="exportMessage" class="export-message">{{ exportMessage }}</p>
3089 </div>
3090 </section>
3091 </template>
3092 </template>
3093 <style scoped>
3094 .analysis-file-picker {
3095     display: flex;
3096     align-items: center;
3097     justify-content: space-between;
3098     gap: 18px;
3099     padding: 18px;
3100     border: 1px dashed #99ad9f;
3101     border-radius: 12px;
3102     background: #f5f9f5;
3103     cursor: pointer;
3104 }
3105 .analysis-file-picker input {
3106     position: absolute;
3107     width: 1px;
3108     height: 1px;
3109     opacity: 0;
3110 }
3111 .analysis-file-picker span {
3112     font-weight: 800;
3113 }
3114 .analysis-file-picker small {

```

```
3115   color: #728078;
3116 }
3117 .analysis-file-summary,
3118 .model-metrics,
3119 .correlation-result {
3120   display: flex;
3121   flex-wrap: wrap;
3122   gap: 10px;
3123 }
3124 .analysis-file-summary span,
3125 .binary-mapping span {
3126   padding: 7px 10px;
3127   border-radius: 8px;
3128   background: #eaf1eb;
3129   font-size: 0.8rem;
3130   font-weight: 700;
3131 }
3132 .analysis-section-title {
3133   margin: 26px 0 14px;
3134 }
3135 .method-grid {
3136   display: grid;
3137   grid-template-columns: repeat(5, 1fr);
3138   gap: 9px;
3139 }
3140 .method-card {
3141   min-height: 104px;
3142   padding: 14px;
3143   border: 1px solid #dce3de;
3144   border-radius: 12px;
3145   background: #fafcf9;
3146   cursor: pointer;
3147 }
3148 .method-card:has(input:checked) {
3149   border-color: #3f7556;
3150   background: #edf6ef;
3151 }
3152 .method-card input {
3153   accent-color: #315e45;
3154 }
3155 .method-card strong,
3156 .method-card small {
3157   display: block;
3158   margin-top: 7px;
3159 }
3160 .method-card small {
3161   color: #718077;
3162   line-height: 1.35;
3163 }
3164 .variable-grid {
```

```
3165     display: grid;
3166     grid-template-columns: repeat(2, 1fr);
3167     gap: 14px;
3168     margin: 22px 0;
3169 }
3170 .variable-grid > label > span,
3171 .independent-selector > span {
3172     display: block;
3173     margin-bottom: 8px;
3174     font-size: 0.8rem;
3175     font-weight: 800;
3176 }
3177 .variable-grid select {
3178     width: 100%;
3179     padding: 11px;
3180     border: 1px solid #ccd7d0;
3181     border-radius: 9px;
3182     color: #25372d;
3183     background: white;
3184 }
3185 .independent-selector > div {
3186     display: flex;
3187     flex-wrap: wrap;
3188     gap: 7px;
3189 }
3190 .independent-selector label {
3191     padding: 9px 11px;
3192     border-radius: 8px;
3193     background: #f1f4f1;
3194     cursor: pointer;
3195 }
3196 .analysis-warning {
3197     color: #9a5c12;
3198 }
3199 .correlation-result article,
3200 .model-metrics article {
3201     min-width: 160px;
3202     padding: 18px;
3203     border-radius: 12px;
3204     background: #f1f6f2;
3205 }
3206 .correlation-result span,
3207 .correlation-result strong,
3208 .correlation-result small,
3209 .model-metrics span,
3210 .model-metrics strong {
3211     display: block;
3212 }
3213 .correlation-result strong,
3214 .model-metrics strong {
```

```
3215     margin-top: 7px;
3216     font-size: 1.35rem;
3217 }
3218 .correlation-result small {
3219     margin-top: 5px;
3220     color: #627068;
3221 }
3222 .model-metrics {
3223     margin-bottom: 22px;
3224 }
3225 .binary-mapping {
3226     margin-bottom: 18px;
3227 }
3228 .analysis-disclaimer {
3229     margin: 20px 0 0;
3230     padding: 12px;
3231     border-radius: 9px;
3232     color: #6f6756;
3233     background: #f8f4e8;
3234     font-size: 0.78rem;
3235 }
3236 .analysis-export-panel {
3237     margin-top: 18px;
3238 }
3239 .diagnostics-panel {
3240     margin-bottom: 20px;
3241     padding: 16px;
3242     border: 1px solid #b9d8c2;
3243     border-radius: 11px;
3244     background: #edf7ef;
3245 }
3246 .diagnostics-panel.invalid {
3247     border-color: #e1a39b;
3248     background: #fff0ed;
3249 }
3250 .diagnostics-heading {
3251     display: flex;
3252     align-items: flex-start;
3253     justify-content: space-between;
3254     gap: 16px;
3255 }
3256 .diagnostics-heading strong,
3257 .diagnostics-heading small {
3258     display: block;
3259 }
3260 .diagnostics-heading small {
3261     margin-top: 4px;
3262     color: #617067;
3263 }
3264 .diagnostic-values,
```

```
3265 .vif-list {
3266     display: flex;
3267     flex-wrap: wrap;
3268     gap: 7px;
3269 }
3270 .diagnostic-values span,
3271 .vif-list span {
3272     padding: 6px 9px;
3273     border-radius: 7px;
3274     background: rgb(255 255 255 / 72%);
3275     font-size: 0.76rem;
3276     font-weight: 700;
3277 }
3278 .diagnostics-panel ul {
3279     margin-bottom: 8px;
3280     padding-left: 20px;
3281 }
3282 .diagnostics-panel li + li {
3283     margin-top: 6px;
3284 }
3285 .diagnostics-panel details {
3286     margin-top: 10px;
3287 }
3288 .diagnostics-panel summary {
3289     cursor: pointer;
3290     font-weight: 700;
3291 }
3292 .vif-list {
3293     margin-top: 10px;
3294 }
3295 @media (max-width: 900px) {
3296     .method-grid {
3297         grid-template-columns: repeat(2, 1fr);
3298     }
3299 }
3300 @media (max-width: 640px) {
3301     .analysis-file-picker,
3302     .variable-grid {
3303         align-items: flex-start;
3304         grid-template-columns: 1fr;
3305     }
3306     .analysis-file-picker {
3307         flex-direction: column;
3308     }
3309 }
3310 </style>
3311 <!-- Source file: frontend/src/components/SpatialWorkspace.vue -->
3312 <script setup lang="ts">
3313 import { computed, ref, watch } from 'vue'
3314 import { exportSpatialAnalysis, previewFile, runSpatialAnalysis } from '@services/api'
```

```

3315 import type {
3316   CoordinateType,
3317   ExportFormat,
3318   FilePreviewResponse,
3319   SpatialAnalysisResponse,
3320   SpatialMethod,
3321 } from '@types/analysis'
3322 const methods: Array<{ value: SpatialMethod; label: string; description: string }> = [
3323   { value: 'moran', label: 'Moran's I', description: '检验一个变量的全局空间自相关' },
3324   { value: 'slm', label: 'SLM / SAR', description: '包含因变量空间滞后项 Wy' },
3325   { value: 'sem', label: 'SEM', description: '误差项存在空间相关结构' },
3326   { value: 'sdm', label: 'SDM', description: '同时包含 Wy 与自变量空间滞后 WX' },
3327   { value: 'gwr', label: 'GWR', description: '估计随空间位置变化的局部系数' },
3328 ]
3329 const selectedFile = ref<File | null>(null)
3330 const dataset = ref<FilePreviewResponse | null>(null)
3331 const selectedMethod = ref<SpatialMethod>('moran')
3332 const coordinateType = ref<CoordinateType>('geographic')
3333 const xColumn = ref('')
3334 const yColumn = ref('')
3335 const dependentColumn = ref('')
3336 const independentColumns = ref<string[]>([])
3337 const neighbors = ref(8)
3338 const result = ref<SpatialAnalysisResponse | null>(null)
3339 const errorMessage = ref('')
3340 const isReading = ref(false)
3341 const isRunning = ref(false)
3342 const exportingFormat = ref<ExportFormat | ''>('')
3343 const exportMessage = ref('')
3344 const numericColumns = computed(() =>
3345   (dataset.value?.quality.columns ?? [])
3346     .filter(
3347       (column) =>
3348         !column.mixed_types &&
3349         column.detected_types.length === 1 &&
3350         column.detected_types[0]?.type === 'number',
3351     )
3352     .map((column) => column.name),
3353 )
3354 const isMoran = computed(() => selectedMethod.value === 'moran')
3355 const availableIndependentColumns = computed(() =>
3356   numericColumns.value.filter(
3357     (column) =>
3358       column !== dependentColumn.value && column !== xColumn.value && column !== yColumn.value,
3359   ),
3360 )
3361 const canRun = computed(
3362   () =>
3363     Boolean(
3364       selectedFile.value &&

```

```

3365     xColumn.value &&
3366     yColumn.value &&
3367     xColumn.value !== yColumn.value &&
3368     dependentColumn.value &&
3369     (isMoran.value || independentColumns.value.length),
3370   ) &&
3371   neighbors.value >= 1 &&
3372   neighbors.value < (dataset.value?.row_count ?? 1),
3373 )
3374 const localPreviewColumns = computed(() => Object.keys(result.value?.gwr?.local_preview[0] ?? {}))
3375 function detectColumn(columns: string[], candidates: string[]) {
3376   const normalizedCandidates = new Set(candidates.map((candidate) => candidate.toLowerCase()))
3377   return (
3378     columns.find((column) => normalizedCandidates.has(column.trim().toLowerCase())) ?? ''
3379   )
3380 }
3381 function detectCoordinateColumns(columns: string[]) {
3382   const longitude = detectColumn(columns, ['longitude', 'lon', 'lng', '经度'])
3383   const latitude = detectColumn(columns, ['latitude', 'lat', '纬度'])
3384   if (longitude && latitude) {
3385     coordinateType.value = 'geographic'
3386     xColumn.value = longitude
3387     yColumn.value = latitude
3388     return
3389   }
3390   const projectedX = detectColumn(columns, ['x', 'x_coord', 'x_coordinate', '投影x', '横坐标'])
3391   const projectedY = detectColumn(columns, ['y', 'y_coord', 'y_coordinate', '投影y', '纵坐标'])
3392   if (projectedX && projectedY) {
3393     coordinateType.value = 'projected'
3394     xColumn.value = projectedX
3395     yColumn.value = projectedY
3396   }
3397 }
3398 watch([selectedMethod, xColumn, yColumn, dependentColumn, coordinateType, neighbors], () => {
3399   result.value = null
3400   errorMessage.value = ''
3401   exportMessage.value = ''
3402   independentColumns.value = independentColumns.value.filter(
3403     (column) =>
3404       column !== dependentColumn.value && column !== xColumn.value && column !== yColumn.value,
3405   )
3406 })
3407 async function onFileSelected(event: Event) {
3408   const input = event.target as HTMLInputElement
3409   const file = input.files?.[0]
3410   if (!file) return
3411   selectedFile.value = file
3412   dataset.value = null
3413   result.value = null
3414   xColumn.value = ''

```

```

3415   yColumn.value = ''
3416   dependentColumn.value = ''
3417   independentColumns.value = []
3418   errorMessage.value = ''
3419   isReading.value = true
3420   try {
3421     dataset.value = await previewFile(file)
3422     neighbors.value = Math.min(8, Math.max(1, dataset.value.row_count - 1))
3423     detectCoordinateColumns(numericColumns.value)
3424   } catch (error) {
3425     selectedFile.value = null
3426     errorMessage.value = error instanceof Error ? error.message : '读取字段失败。'
3427   } finally {
3428     isReading.value = false
3429   }
3430 }
3431 async function executeSpatialAnalysis() {
3432   if (!selectedFile.value || !canRun.value) return
3433   isRunning.value = true
3434   result.value = null
3435   errorMessage.value = ''
3436   try {
3437     result.value = await runSpatialAnalysis(selectedFile.value, currentOptions())
3438   } catch (error) {
3439     errorMessage.value = error instanceof Error ? error.message : '空间分析失败。'
3440   } finally {
3441     isRunning.value = false
3442   }
3443 }
3444 function currentOptions() {
3445   return {
3446     method: selectedMethod.value,
3447     coordinateType: coordinateType.value,
3448     xColumn: xColumn.value,
3449     yColumn: yColumn.value,
3450     dependentColumn: dependentColumn.value,
3451     independentColumns: isMoran.value ? [] : independentColumns.value,
3452     neighbors: neighbors.value,
3453   }
3454 }
3455 async function downloadSpatialResult(outputFormat: ExportFormat) {
3456   if (!selectedFile.value || !result.value) return
3457   exportingFormat.value = outputFormat
3458   errorMessage.value = ''
3459   exportMessage.value = ''
3460   try {
3461     const filename = await exportSpatialAnalysis(
3462       selectedFile.value,
3463       currentOptions(),
3464       outputFormat,

```

```

3465     )
3466     exportMessage.value = `已导出 ${filename}`
3467   } catch (error) {
3468     errorMessage.value = error instanceof Error ? error.message : '空间结果导出失败。'
3469   } finally {
3470     exportingFormat.value = ''
3471   }
3472 }
3473 function formatNumber(value: number | null | undefined) {
3474   if (value === null || value === undefined) return '—'
3475   if (value !== 0 && Math.abs(value) < 0.0001) return value.toExponential(3)
3476   return value.toFixed(6)
3477 }
3478 function moranInterpretation(value: number | null) {
3479   if (value === null) return '无法判断'
3480   if (value > 0) return '正空间自相关（相似值趋于聚集）'
3481   if (value < 0) return '负空间自相关（相异值趋于相邻）'
3482   return '接近随机空间分布'
3483 }
3484 </script>
3485 <template>
3486   <section class="panel spatial-intro">
3487     <div class="section-heading">
3488       <div>
3489         <p class="step-label">SPATIAL</p>
3490         <h2>空间分析</h2>
3491       </div>
3492       <p>支持经纬度或投影 X/Y 坐标，使用 K 近邻构建空间权重</p>
3493     </div>
3494     <label class="spatial-file-picker">
3495       <input type="file" accept=".csv,.xlsx" @change="onFileSelected" />
3496       <span>{{ isReading ? '正在读取字段...' : '选择空间数据' }}</span>
3497       <small>{{ selectedFile?.name ?? '每一行应代表一个带坐标的空间观测对象' }}</small>
3498     </label>
3499     <p v-if="errorMessage" class="error-message">{{ errorMessage }}</p>
3500   </section>
3501   <template v-if="dataset">
3502     <section class="panel">
3503       <div class="spatial-summary-tags">
3504         <span>{{ dataset.filename }}</span>
3505         <span>{{ dataset.row_count }} 行</span>
3506         <span>{{ numericColumns.length }} 个可用数值列</span>
3507       </div>
3508       <h3 class="spatial-section-title">1. 选择空间分析方法</h3>
3509       <div class="spatial-method-grid">
3510         <label v-for="method in methods" :key="method.value" class="spatial-method-card">
3511           <input v-model="selectedMethod" type="radio" :value="method.value" />
3512           <strong>{{ method.label }}</strong>
3513           <small>{{ method.description }}</small>
3514         </label>

```

```

3515 </div>
3516 <h3 class="spatial-section-title">2. 设置坐标与空间权重</h3>
3517 <div class="coordinate-type-picker">
3518   <label>
3519     <input v-model="coordinateType" type="radio" value="geographic" />
3520     经纬度 (WGS84)
3521   </label>
3522   <label>
3523     <input v-model="coordinateType" type="radio" value="projected" />
3524     投影坐标 X / Y
3525   </label>
3526 </div>
3527 <div class="spatial-variable-grid">
3528   <label>
3529     <span>{{ coordinateType === 'geographic' ? '经度列' : 'X 坐标列' }}</span>
3530     <select v-model="xColumn">
3531       <option value="" disabled>请选择数值列</option>
3532       <option v-for="column in numericColumns" :key="column" :value="column">{{ column }}</option>
3533     </select>
3534   </label>
3535   <label>
3536     <span>{{ coordinateType === 'geographic' ? '纬度列' : 'Y 坐标列' }}</span>
3537     <select v-model="yColumn">
3538       <option value="" disabled>请选择数值列</option>
3539       <option v-for="column in numericColumns" :key="column" :value="column">{{ column }}</option>
3540     </select>
3541   </label>
3542   <label>
3543     <span>K 近邻数</span>
3544     <input v-model.number="neighbors" type="number" min="1" :max="dataset.row_count - 1" />
3545     <small>默认 8; 每个观测连接最近的 K 个观测</small>
3546   </label>
3547 </div>
3548 <h3 class="spatial-section-title">3. 选择分析变量</h3>
3549 <div class="spatial-variable-grid">
3550   <label>
3551     <span>{{ isMoran ? '空间自相关变量' : '因变量 Y' }}</span>
3552     <select v-model="dependentColumn">
3553       <option value="" disabled>请选择数值列</option>
3554       <option v-for="column in numericColumns" :key="column" :value="column">{{ column }}</option>
3555     </select>
3556   </label>
3557   <div v-if="!isMoran" class="spatial-independent-selector">
3558     <span>自变量 X (可多选)</span>
3559     <div>
3560       <label v-for="column in availableIndependentColumns" :key="column">
3561         <input v-model="independentColumns" type="checkbox" :value="column" />
3562         {{ column }}
3563       </label>
3564     </div>

```

```

3565     </div>
3566 </div>
3567 <p class="spatial-note">
3568     缺失坐标或模型变量的行会被排除。经纬度按球面邻近关系处理；投影坐标按平面距离处理。
3569 </p>
3570 <button class="primary-action" :disabled="isRunning || !canRun" @click="executeSpatialAnalysis">
3571     {{ isRunning ? (selectedMethod === 'gwr' ? '正在选择带宽并拟合...' : '正在计算...') : '运行空间分析' }}
3572 </button>
3573 </section>
3574 <section v-if="result" class="panel spatial-result">
3575     <div class="section-heading">
3576         <div>
3577             <p class="step-label">SPATIAL RESULT</p>
3578             <h2>空间分析结果</h2>
3579         </div>
3580         <p>有效样本 {{ result.observations }}，排除缺失行 {{ result.dropped_rows }}</p>
3581     </div>
3582     <div
3583         class="spatial-diagnostics"
3584         :class="{ invalid: !result.diagnostics.valid_inference }"
3585     >
3586         <div class="spatial-diagnostics-heading">
3587             <div>
3588                 <strong>
3589                     {{ result.diagnostics.valid_inference ? '模型推断有效' : '模型推断不可靠' }}
3590                 </strong>
3591                 <small>
3592                     {{ result.diagnostics.converged ? '计算已正常完成' : '模型未收敛' }}
3593                 </small>
3594             </div>
3595             <div class="spatial-diagnostic-values">
3596                 <span v-if="result.diagnostics.condition_number !== null">
3597                     标准化条件数 {{ formatNumber(result.diagnostics.condition_number) }}
3598                 </span>
3599                 <span v-if="result.diagnostics.raw_condition_number !== null">
3600                     原始条件数 {{ formatNumber(result.diagnostics.raw_condition_number) }}
3601                 </span>
3602                 <span v-if="result.diagnostics.max_vif !== null">
3603                     最大 VIF {{ formatNumber(result.diagnostics.max_vif) }}
3604                 </span>
3605                 <span v-if="result.diagnostics.scale_ratio !== null">
3606                     尺度比 {{ formatNumber(result.diagnostics.scale_ratio) }}
3607                 </span>
3608             </div>
3609         </div>
3610         <ul v-if="result.diagnostics.warnings.length">
3611             <li v-for="warning in result.diagnostics.warnings" :key="warning">{{ warning }}</li>
3612         </ul>
3613         <p v-else>未检测到收敛、非有限数值、重复坐标或明显共线性问题。</p>
3614         <details v-if="result.diagnostics.vif.length">

```

```

3615     <summary>查看各变量 VIF</summary>
3616     <div class="spatial-vif-list">
3617         <span v-for="item in result.diagnostics.vif" :key="item.column">
3618             {{ item.column }}: {{ formatNumber(item.vif) }}
3619         </span>
3620     </div>
3621 </details>
3622 </div>
3623 <div class="spatial-summary-tags weight-summary">
3624     <span>KNN: {{ result.weights.neighbors }} 个邻居</span>
3625     <span>行标准化权重</span>
3626     <span>连通分量: {{ result.weights.components }}</span>
3627 </div>
3628 <div v-if="result.moran" class="spatial-metric-grid">
3629     <article><span>Moran's I</span><strong>{{ formatNumber(result.moran.i) }}</strong></article>
3630     <article><span>期望值</span><strong>{{ formatNumber(result.moran.expected_i) }}</strong></article>
3631     <article><span>Z 值</span><strong>{{ formatNumber(result.moran.z_score) }}</strong></article>
3632     <article><span>置换检验 p 值</span><strong>{{ formatNumber(result.moran.p_permutation) }}</strong></article>
3633     <p class="moran-interpretation">{{ moranInterpretation(result.moran.i) }}</p>
3634 </div>
3635 <template v-if="result.regression">
3636     <div class="spatial-metric-grid">
3637         <article><span>伪 R2</span><strong>{{ formatNumber(result.regression.metrics.pseudo_r_squared) }}</strong></article>
3638         <article><span>AIC</span><strong>{{ formatNumber(result.regression.metrics.aic) }}</strong></article>
3639         <article><span>BIC</span><strong>{{ formatNumber(result.regression.metrics.bic) }}</strong></article>
3640         <article v-if="result.regression.metrics.rho !== null"><span> $\rho$  (空间滞后)</span><strong>{{ formatNumber(result.regression.metrics.rho) }}</strong></article>
3641         <article v-if="result.regression.metrics.lambda !== null"><span> $\lambda$  (空间误差)</span><strong>{{ formatNumber(result.regression.metrics.lambda) }}</strong></article>
3642     </div>
3643     <div class="quality-table-wrap">
3644         <table class="quality-table">
3645             <thead><tr><th>变量</th><th>系数</th><th>标准误</th><th>Z 值</th><th>p 值</th><th>95% 置信区间</th></tr></thead>
3646             <tbody>
3647                 <tr v-for="coefficient in result.regression.coefficients" :key="coefficient.term">
3648                     <td class="column-name">{{ coefficient.term }}</td>
3649                     <td>{{ formatNumber(coefficient.estimate) }}</td>
3650                     <td>{{ formatNumber(coefficient.standard_error) }}</td>
3651                     <td>{{ formatNumber(coefficient.statistic) }}</td>
3652                     <td>{{ formatNumber(coefficient.p_value) }}</td>
3653                     <td>{{ formatNumber(coefficient.confidence_low) }} ~ {{ formatNumber(coefficient.confidence_high) }}</td>
3654                 </tr>
3655             </tbody>
3656         </table>
3657     </div>
3658 <template v-if="result.regression.spatial_impacts.length">
3659     <h3 class="spatial-subheading">空间效应分解</h3>
3660     <p class="spatial-result-note">
3661         直接效应表示本地影响, 间接效应表示经空间邻居传播的溢出影响, 总效应为两者之和。
3662     </p>
3663     <div class="quality-table-wrap">
3664         <table class="quality-table">

```

```

3665         <thead>
3666             <tr><th>变量</th><th>直接效应</th><th>间接效应</th><th>总效应</th></tr>
3667         </thead>
3668         <tbody>
3669             <tr v-for="impact in result.regression.spatial_impacts" :key="impact.term">
3670                 <td class="column-name">{{ impact.term }}</td>
3671                 <td>{{ formatNumber(impact.direct) }}</td>
3672                 <td>{{ formatNumber(impact.indirect) }}</td>
3673                 <td>{{ formatNumber(impact.total) }}</td>
3674             </tr>
3675         </tbody>
3676     </table>
3677 </div>
3678 </template>
3679 </template>
3680 <template v-if="result.gwr">
3681     <div class="spatial-metric-grid">
3682         <article><span>自适应带宽</span><strong>{{ formatNumber(result.gwr.bandwidth) }}</strong></article>
3683         <article><span>R2</span><strong>{{ formatNumber(result.gwr.metrics.r_squared) }}</strong></article>
3684         <article><span>调整 R2</span><strong>{{ formatNumber(result.gwr.metrics.adjusted_r_squared) }}</strong></article>
3685         <article><span>AICc</span><strong>{{ formatNumber(result.gwr.metrics.aicc) }}</strong></article>
3686     </div>
3687     <h3>局部系数分布</h3>
3688     <div class="quality-table-wrap">
3689         <table class="quality-table">
3690             <thead><tr><th>变量</th><th>均值</th><th>标准差</th><th>最小值</th><th>中位数</th><th>最大值</th></tr></thead>
3691             <tbody>
3692                 <tr v-for="coefficient in result.gwr.coefficient_summaries" :key="coefficient.term">
3693                     <td class="column-name">{{ coefficient.term }}</td>
3694                     <td>{{ formatNumber(coefficient.mean) }}</td>
3695                     <td>{{ formatNumber(coefficient.standard_deviation) }}</td>
3696                     <td>{{ formatNumber(coefficient.minimum) }}</td>
3697                     <td>{{ formatNumber(coefficient.median) }}</td>
3698                     <td>{{ formatNumber(coefficient.maximum) }}</td>
3699                 </tr>
3700             </tbody>
3701         </table>
3702     </div>
3703     <h3 class="local-preview-heading">局部结果预览（前 {{ result.gwr.local_preview.length }} 行）</h3>
3704     <div class="quality-table-wrap">
3705         <table class="quality-table">
3706             <thead><tr><th v-for="column in localPreviewColumns" :key="column">{{ column }}</th></tr></thead>
3707             <tbody>
3708                 <tr v-for="(row, index) in result.gwr.local_preview" :key="index">
3709                     <td v-for="column in localPreviewColumns" :key="column">{{ formatNumber(row[column]) }}</td>
3710                 </tr>
3711             </tbody>
3712         </table>
3713     </div>
3714 </template>

```

```

3715 <template v-if="result.residual_moran">
3716   <h3 class="spatial-subheading">当前模型残差 Moran</h3>
3717   <div class="spatial-metric-grid">
3718     <article>
3719       <span>残差 Moran's I</span>
3720       <strong>{{ formatNumber(result.residual_moran.i) }}</strong>
3721     </article>
3722     <article>
3723       <span>Z 值</span>
3724       <strong>{{ formatNumber(result.residual_moran.z_score) }}</strong>
3725     </article>
3726     <article>
3727       <span>置换检验 p 值</span>
3728       <strong>{{ formatNumber(result.residual_moran.p_permutation) }}</strong>
3729     </article>
3730   </div>
3731   <p
3732     class="residual-interpretation"
3733     :class="{ warning: (result.residual_moran.p_permutation ?? 1) < 0.05 }"
3734   >
3735     {{
3736       (result.residual_moran.p_permutation ?? 1) < 0.05
3737       ? '残差仍存在显著空间自相关，当前模型可能尚未充分解释空间结构。'
3738       : '残差 Moran 未达到 0.05 显著水平，未发现明显的剩余空间自相关。'
3739     }}
3740   </p>
3741 </template>
3742 <template v-if="result.model_selection">
3743   <h3 class="spatial-subheading">空间模型选择诊断</h3>
3744   <p class="model-recommendation">{{ result.model_selection.recommendation }}</p>
3745   <div class="quality-table-wrap">
3746     <table class="quality-table">
3747       <thead><tr><th>检验</th><th>统计量</th><th>p 值</th><th>判断</th></tr></thead>
3748       <tbody>
3749         <tr v-for="test in result.model_selection.tests" :key="test.name">
3750           <td class="column-name">{{ test.name }}</td>
3751           <td>{{ formatNumber(test.statistic) }}</td>
3752           <td>{{ formatNumber(test.p_value) }}</td>
3753           <td>{{ (test.p_value ?? 1) < 0.05 ? '显著' : '不显著' }}</td>
3754         </tr>
3755       </tbody>
3756     </table>
3757   </div>
3758   <p class="spatial-result-note">
3759     模型选择提示以基础 OLS 的稳健 LM 检验为主；还应结合研究理论、空间权重敏感性和模型拟合指标。
3760   </p>
3761   <ul v-if="result.model_selection.warnings.length" class="selection-warnings">
3762     <li v-for="warning in result.model_selection.warnings" :key="warning">
3763       诊断运行警告：{{ warning }}
3764     </li>

```

```

3765     </ul>
3766 </template>
3767 <p class="spatial-disclaimer">
3768     空间模型结果依赖坐标系、空间权重与邻居数设定。正式结论应比较不同 K 值，并检查残差空间自相关和模型假设。
3769 </p>
3770 <div class="export-panel spatial-export-panel">
3771     <div>
3772         <h3>导出空间分析结果</h3>
3773         <p v-if="result.gwr">
3774             GWR 导出会重新拟合模型，并写入全部 {{ result.gwr.local_result_count }} 行局部结果。
3775             局部 p_value_unadjusted 为根据 t 值计算、未经多重检验校正的双侧近似 p 值。
3776         </p>
3777         <p v-else>XLSX 同时包含模型概览、诊断、VIF 和结果明细。</p>
3778     </div>
3779     <div class="export-actions">
3780         <button :disabled="!!exportingFormat" @click="downloadSpatialResult('csv')">
3781             {{ exportingFormat === 'csv' ? '导出中...' : '导出 CSV' }}
3782         </button>
3783         <button
3784             class="xlsx-button"
3785             :disabled="!!exportingFormat"
3786             @click="downloadSpatialResult('xlsx')">
3787             >
3788             {{ exportingFormat === 'xlsx' ? '导出中...' : '导出 XLSX' }}
3789         </button>
3790     </div>
3791     <p v-if="exportMessage" class="export-message">{{ exportMessage }}</p>
3792 </div>
3793 </section>
3794 </template>
3795 </template>
3796 <style scoped>
3797 .spatial-file-picker {
3798     display: flex;
3799     align-items: center;
3800     justify-content: space-between;
3801     gap: 18px;
3802     padding: 18px;
3803     border: 1px dashed #98aab4;
3804     border-radius: 12px;
3805     background: #f3f8fa;
3806     cursor: pointer;
3807 }
3808 .spatial-file-picker input { position: absolute; width: 1px; height: 1px; opacity: 0; }
3809 .spatial-file-picker span { font-weight: 800; }
3810 .spatial-file-picker small { color: #667983; }
3811 .spatial-summary-tags, .coordinate-type-picker, .spatial-metric-grid { display: flex; flex-wrap: wrap; gap: 10px; }
3812 .spatial-summary-tags span { padding: 7px 10px; border-radius: 8px; background: #e9f1f4; font-size: .8rem; font-weight: 700; }
3813 .spatial-section-title { margin: 26px 0 14px; }
3814 .spatial-method-grid { display: grid; grid-template-columns: repeat(5, 1fr); gap: 9px; }

```

```

3815 .spatial-method-card { min-height: 104px; padding: 14px; border: 1px solid #d8e1e5; border-radius: 12px; background: #fafcf5; cursor: pointer; }
3816 .spatial-method-card:has(input:checked) { border-color: #3d7186; background: #eaf4f8; }
3817 .spatial-method-card input, .coordinate-type-picker input { accent-color: #32677d; }
3818 .spatial-method-card strong, .spatial-method-card small { display: block; margin-top: 7px; }
3819 .spatial-method-card small { color: #6d7d84; line-height: 1.35; }
3820 .coordinate-type-picker label { padding: 10px 13px; border-radius: 9px; background: #eef3f5; cursor: pointer; }
3821 .spatial-variable-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 14px; margin: 16px 0 22px; }
3822 .spatial-variable-grid > label > span, .spatial-independent-selector > span { display: block; margin-bottom: 8px; font-size: .8rem; font-weight: 800; }
3823 .spatial-variable-grid select, .spatial-variable-grid input[type='number'] { width: 100%; padding: 11px; border: 1px solid #cad7dc; border-radius: 9px; color: #24363d; background: white; }
3824 .spatial-variable-grid small { display: block; margin-top: 6px; color: #74838a; }
3825 .spatial-independent-selector { grid-column: span 2; }
3826 .spatial-independent-selector > div { display: flex; flex-wrap: wrap; gap: 7px; }
3827 .spatial-independent-selector label { padding: 9px 11px; border-radius: 8px; background: #eef3f5; cursor: pointer; }
3828 .spatial-note, .spatial-disclaimer { margin: 16px 0; padding: 12px; border-radius: 9px; color: #635f55; background: #f7f3e7; font-size: .8rem; }
3829 .weight-summary { margin-bottom: 18px; }
3830 .spatial-metric-grid article { min-width: 155px; padding: 17px; border-radius: 12px; background: #edf4f6; }
3831 .spatial-metric-grid article span, .spatial-metric-grid article strong { display: block; }
3832 .spatial-metric-grid article strong { margin-top: 7px; font-size: 1.3rem; }
3833 .moran-interpretation { width: 100%; font-weight: 700; color: #3d6271; }
3834 .local-preview-heading { margin-top: 24px; }
3835 .spatial-disclaimer { margin-top: 20px; }
3836 .spatial-subheading { margin: 24px 0 10px; }
3837 .spatial-result-note { color: #68777d; font-size: .8rem; }
3838 .residual-interpretation, .model-recommendation { padding: 11px 13px; border-radius: 9px; background: #edf6ef; font-weight: 700; }
3839 .residual-interpretation.warning { color: #8c3e34; background: #fff0ed; }
3840 .model-recommendation { color: #345e70; background: #eaf4f8; }
3841 .selection-warnings { color: #8c5b24; font-size: .78rem; }
3842 .spatial-export-panel { margin-top: 18px; }
3843 .spatial-diagnostics { margin-bottom: 18px; padding: 16px; border: 1px solid #b6d3dc; border-radius: 11px; background: #edf7fa; }
3844 .spatial-diagnostics.invalid { border-color: #e1a39b; background: #fff0ed; }
3845 .spatial-diagnostics-heading { display: flex; align-items: flex-start; justify-content: space-between; gap: 16px; }
3846 .spatial-diagnostics-heading strong, .spatial-diagnostics-heading small { display: block; }
3847 .spatial-diagnostics-heading small { margin-top: 4px; color: #64757c; }
3848 .spatial-diagnostic-values, .spatial-vif-list { display: flex; flex-wrap: wrap; gap: 7px; }
3849 .spatial-diagnostic-values span, .spatial-vif-list span { padding: 6px 9px; border-radius: 7px; background: rgb(255 255 255 / 72%); font-size: .76rem; font-weight: 700; }
3850 .spatial-diagnostics ul { margin-bottom: 8px; padding-left: 20px; }
3851 .spatial-diagnostics li + li { margin-top: 6px; }
3852 .spatial-diagnostics details { margin-top: 10px; }
3853 .spatial-diagnostics summary { cursor: pointer; font-weight: 700; }
3854 .spatial-vif-list { margin-top: 10px; }
3855 @media (max-width: 900px) {
3856   .spatial-method-grid { grid-template-columns: repeat(2, 1fr); }
3857   .spatial-variable-grid { grid-template-columns: 1fr 1fr; }
3858 }
3859 @media (max-width: 640px) {
3860   .spatial-file-picker { align-items: flex-start; flex-direction: column; }
3861   .spatial-variable-grid { grid-template-columns: 1fr; }
3862   .spatial-independent-selector { grid-column: auto; }
3863 }
3864 </style>

```